CO4015 MComp Project

Dissertation

“Tracer Racers” – An Implementation of the “Racetrack” Game

Muhammad Mogra

27th April 2026

School of Computing and Mathematical Sciences
University of Leicester

Declaration

All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stages, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s).

Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s).

I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.

Name: MUHAMMAD MOGRA

Signed: 

Abstract

The *Racetrack* game is a pencil-and-paper game in which players must race around a track in turns with vectors. A few computer games based on the *Racetrack* concept exist, but they tend to be simple, with minimal gameplay features, or each has features that others lack, which would be interesting to combine. This project aims to recreate and build upon the *Racetrack* game in these areas, producing a web application built with HTML, JavaScript, Node.js, MongoDB and Firebase Hosting & Authentication. The application extends on the features with spline-based track building, leaderboards, online capabilities, AI racers, and a selection of car types. The project managed to fulfil all original objectives using Node.js and Firebase, but could be improved in terms of security and gameplay.

Contents

1. Introduction 7
   1.1. Background 7
   1.2. Project Aims 7
   1.3. Project Objectives 7
2. Existing Games 8
   2.1. Main Reference Points 8
   2.2. Primary Objectives Relations 8
      2.2.1. Primary Objective 4.1 9
      2.2.2. Primary Objective 4.2 10
      2.2.3. Primary Objective 4.3 11
      2.2.4. Primary Objective 4.4 11
      2.2.5. Primary Objective 4.5 12
   2.3. Secondary Objectives Relations 12
      2.3.1. Secondary Objective 1 12
      2.3.2. Secondary Objective 2 13
      2.3.3. Secondary Objective 3 14
   2.4. Other Games 14
3. Chosen Technologies 15
4. Application Requirements 16
   4.1. Requirements Glossary 16
   4.2. Requirements List 17
5. Design and Architecture 20
   5.1. Files and Execution 20
   5.2. Pages and Client Scripts 22
   5.3. Component Integration and Routing 24
   5.4. Data Structures and Serialisation 27
      5.4.1. Objects and Class Diagrams 27
      5.4.2. MongoDB Data 28
      5.4.3. Firebase Realtime Database Data 30
   5.5. User Login System 31
   5.6. Icons and Styling 32
6. Implementation and Algorithms 32
   6.1. Track Representation 32
      6.1.1. Track Objects 32

6.1.2. Spline Calculations and Track Form 34
6.1.3. Track Visualisation 36
6.2. Track Building 37
6.2.1. Track Builder Overview 38
6.2.2. Visualisation Options 40
6.2.3. Tool Options 40
6.2.4. Manipulation Options 41
6.2.5. Element-Specific Options 41
6.2.6. Toolbar Options 41
6.3. Track Listing 43
6.3.1. Tracks List Overview 43
6.3.2. Track Details 44
6.4. Lobby System 45
6.4.1. Lobby Page Overview 45
6.4.2. Player Details 47
6.4.3. Change Propagation 48
6.4.4. Lobby Lifecycle 49
6.5. Race Gameplay 51
6.5.1. Race Page Overview 52
6.5.2. Player Move Selection 52
6.5.3. Robot Pathfinding and Difficulty 55
6.5.4. Ghost Move Selection 57
6.5.5. Turn System 57
6.5.6. Race Results and Leaderboards 58
7. Testing 60
8. Critical Appraisal & Future Work 60
8.1. Evaluation of Primary Objectives 60
8.2. Primary Objective 4.1 60
8.2.2. Primary Objective 4.3 61
8.2.3. Primary Objective 4.4 61
8.2.4. Primary Objective 4.5 61
8.3. Evaluation of Secondary Objectives 61
8.3.1. Secondary Objective 1 61
8.3.3. Secondary Objective 3 62
8.4. Project Impact 62

8.5. Personal Development 62
8.6. Improvements & Future Work 63
9. Conclusions 65
10. References 65

1. Introduction

1.1. Background

The *Racetrack* game (Gardner, 1973) is a turn-based pencil-and-paper (i.e., 2D) game in which players must race around a track using vectors on a grid, rather than in continuous 3D space. Players can select a cell to move to within their acceleration limit, typically within 1 unit, where the eight neighbours they can select are determined by their current velocity. Selecting a tile then determines the player's subsequent velocity, and the cycle repeats. Players must monitor their velocity and plan their path around without crashing.

1.2. Project Aims

A few computer games based on the *Racetrack* concept exist, but they tend to be simple, with minimal gameplay features, or they each have features others lack that would be interesting to combine. This project aims to recreate and build upon the *Racetrack* game in these areas, discussed in more detail in **Section 2**.

The project is primarily planned to include track-building capabilities, the ability to play tracks and compete locally against other players, and AI opponents to race against, all integrated into a single cohesive application. These features provide a challenging experience for players on custom tracks against other drivers, leading to an engaging experience (Lazzaro, 2009). While the concept of vectors is mathematical, the project focuses more on gameplay than education. Nevertheless, players may find it a useful and engaging tool for learning the concept (Russo, 2021). Secondary features will also be implemented to support online multiplayer capabilities and customisation, enhancing the player experience and enabling connections beyond local play. It may also include varying AI difficulties to ensure they are not too perfect to compete against.

1.3. Project Objectives

Based on the aims described above, the objectives for the project can be split into two sections: the **Primary Objectives**, which must be completed to ensure the project is successful and fulfils the aims; and **Secondary Objectives**, which cover more challenging areas that enhance gameplay, though are not necessary for the project's success.

Primary Objectives

1. Compile game requirements based on the aims
2. Design the game architecture
3. Draw screen and gameplay designs
4. Implement the functionality for gameplay features
    1. Enable players to race around tracks with vectors
    2. Enable players to create tracks
    3. Enable players to manage their tracks
    4. Enable players to race against other players locally
    5. Enable players to race against AI racers
5. Perform tests to ensure all functionality operates as expected

Secondary Objectives

1. Enable players to race against other players online
2. Enable players to race against AI racers of varying difficulty
3. Enable players to personalise their gameplay experience

2. Existing Games

When developing a new application, it is useful to research existing ones in the same area to understand how results have been achieved (Flora & Chande, 2013). In the project's case, this would include how the Racetrack concept has been implemented and any gameplay elements. This provides ideas for features which would be interesting, helpful, or challenging to ensure a positive gameplay experience and a fulfilling project (Lazzaro, 2009). Potential improvements can be considered for the project's version, as some or none of the apps may include certain elements. Nielsen (1994) also proposes 10 design heuristics for developing applications that serve as a guide to help ensure players can use them with minimal effort.

2.1. Main Reference Points

The Google search queries `"racetrack game"` and `"racetrack game vector"` were used to find games to research. While there were a fair number of games, the following were chosen as they are either most like the *Racetrack* concept or contribute significantly to another objective:

1. **Inertia Racetrack:** a simple web-app recreation of the *Racetrack* game, allowing two players to compete locally across ten unique tracks. Players take turns to drive their race cars around the track until a player reaches the finish line (Transum, 2026).
2. **Vector Racer:** another web-based recreation of the *Racetrack* game with similar gameplay to *Inertia Racetrack,* allowing players to compete with a mix of up to six local players or AI drivers with varying difficulties. In addition to gameplay, it introduces track-building capabilities with materials and ranked leaderboards (Boschloo, 2018).
3. **PolyTrack:** a 3D, low-polygon racing game which plays much like most racing games, where racers can drive freely rather than be bound to a grid. Though it does not incorporate the *Racetrack* concept, it has a sophisticated track-building system with diverse parts, leaderboards, track replays, and player personalisation options (Kodub, 2026).

2.2. Primary Objectives Relations

This section compares each game based on the subobjectives of **Primary Objective 4**.

(section continues)

2.2.1. Primary Objective 4.1

"Enable players to race around tracks with vectors"

Inertia Racetrack and *Vector Racer* both conform to the *Racetrack* concept's rules, defined in **Section 1.1**.

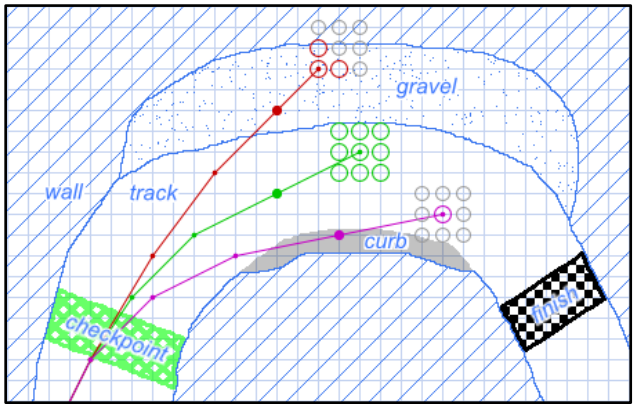


Figure 1 – three racers driving over three material types (gravel, track, and curb) in *Vector Racer*, labelled by name and showing the effect of each one.

Vector Racer visualises velocity selection as shown in **Figure 1**. This allows players to focus more on gameplay than on mental calculations of where their racecar will go, which are required in *Inertia Racetrack*. While manually calculating the race car's next position is a challenge, it is an artificial one that may frustrate or bore players (Hunicke & Chapman, 2004). As a result, it is better to use a visualisation to hasten this process.

In *Vector Racer*, players can reset their car to its last position after crashing or quitting the race, whereas in *Inertia Racetrack*, the game ends entirely. Ending the game may be fine when racing alone, but it can be frustrating when racing others. Allowing players to choose whether to leave without ending the race would make the experience smoother and less annoying for those who would like to continue (Lazzaro, 2009).

In both games, all cars have the same acceleration and velocity limitations, as shown in **Figure 1**, which conform to standard *Racetrack* rules. Having vehicles with varying performance would be intriguing and add a level of skill expression and adaptability to certain tracks, resulting in more refined gameplay (Lazzaro, 2009). Both games also allow cars to be in the same spot. This may be because the *Racetrack* game involves a discrete grid, which makes these overlaps likely, and is something to consider for the application.

(section continues)



Figure 2 – the list of race options available in *Vector Racer* when selecting a track.

Aside from gameplay, both games list the tracks available in the game or created by other players. In addition to game tracks and player tracks, *PolyTrack* also lists the tracks the player has made themselves, which is useful for **Primary Objective 4.2**. Players can view track details from these lists, including names and previews, to decide whether to play a track. In *Vector Racer*, players can also choose race options, such as the number of players and whether to include AI racers, as depicted in **Figure 2**.

2.2.2. Primary Objective 4.2

"Enable players to create tracks"

Both *Vector Racer* and *PolyTrack* include track-building features.

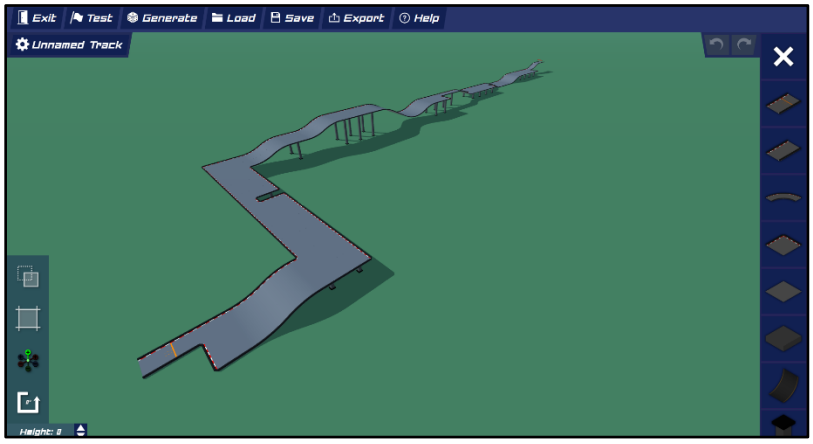


Figure 3 – the track builder from *PolyTrack*, including an auto-generated track and the available track parts (right) grouped by part category (right-most list).

PolyTrack has an in-game track builder with preset components, as shown in **Figure 3**. Meanwhile, *Vector Racer* requires players to upload a colour map, which is converted to a track. *Vector Racer* offers more freedom, since players can draw any shape, while PolyTrack is more well-formed, with physical track parts and an unlimited

building area. The former has strengths in creativity, and the latter has strengths in consistency and ease of placement (Lazzaro, 2009).

PolyTrack also includes an option to auto-generate tracks, made possible with its track parts, which could make the building process easier (Lazzaro, 2009). It also includes options to undo or redo placements, edit a track's details, and save or export it. An in-game editor makes it much easier to assess what the track will look like in-game. The ability to test without uploading the track can save time and frustration, allowing more focus on gameplay (Lazzaro, 2009).

While *Inertia Racetrack* does not include any inherent building tools, its tracks do have decorations. This would be a fun element to include to make tracks look more appealing or for practicality (Lazzaro, 2009), such as directional arrows, so players know where to go.

2.2.3. Primary Objective 4.3

"Enable players to manage their tracks"

PolyTrack is the only game that allows players to manage their tracks. Players can view their tracks, edit their details, open a track for further editing, or delete it entirely.



Figure 4 – the custom tracks section in *PolyTrack* listing a single, player-made test track.

This is useful for players to track, adjust, and share their creations, and the option to remove them helps declutter their list. Players can also search tracks by name or author in each section. This system, as shown in **Figure 4**, would be helpful to include in the project for centralised track management.

2.2.4. Primary Objective 4.4

"Enable players to race against other players locally"

Local racing ensures players can enjoy the game with friends without an Internet connection. Both *Inertia Racetrack* and *Vector Racer* only allow a limited number of local players. Increasing the player limit for races offers more freedom to enjoy the game how they would like (Lazzaro, 2009). However, it is also important to consider that too many players may flood the screen, making the track hard to view.

(section continues)



Figure 5 – the replay system in *PolyTrack* is playing back four players' races alongside each other.

Meanwhile, *PolyTrack* has replays and ghost racing, as demonstrated in **Figure 5**. It would be intriguing to implement replays, but the same level of skill, precision, and timing required in *PolyTrack* would not apply, since it uses arrow keys rather than discrete vectors for movement. As a result, replays would make competition trivial, as players could watch them and copy the optimal path. Moreover, local and online multiplayer will likely cover the competitive spirit that replays have, making them somewhat redundant. However, races against the player's personal best could be a fairer alternative that still allows players to analyse previous races and perform better (Lazzaro, 2009).

2.2.5. Primary Objective 4.5

"Enable players to race against AI racers"

Vector Racer lets players race against AI, allowing players to enjoy the game with some challenge when there is nobody else to play with. However, it automatically fills all remaining player slots when selected. An option to adjust how many AI racers to include would provide more freedom to tailor gameplay to their liking (Lazzaro, 2009), for example, when players may want only a few AI racers.

In general, it is also important to consider AI where there would be considerations for humans, as they may apply to it. For example, as mentioned in **Section 2.2.1**, AI racers could also be given a car type just as players may be able to choose their own. There could also be options to limit which cars AIs can drive to tailor the experience or to target a certain type of race car, providing even more freedom and challenge (Lazzaro, 2009).

2.3. Secondary Objectives Relations

This section assesses and compares each game based on each **Secondary Objective**.

2.3.1. Secondary Objective 1

"Enable players to race against players online"

Interestingly, none of the three games implements a direct form of online multiplayer, which is a new feature the project can incorporate.

(section continues)

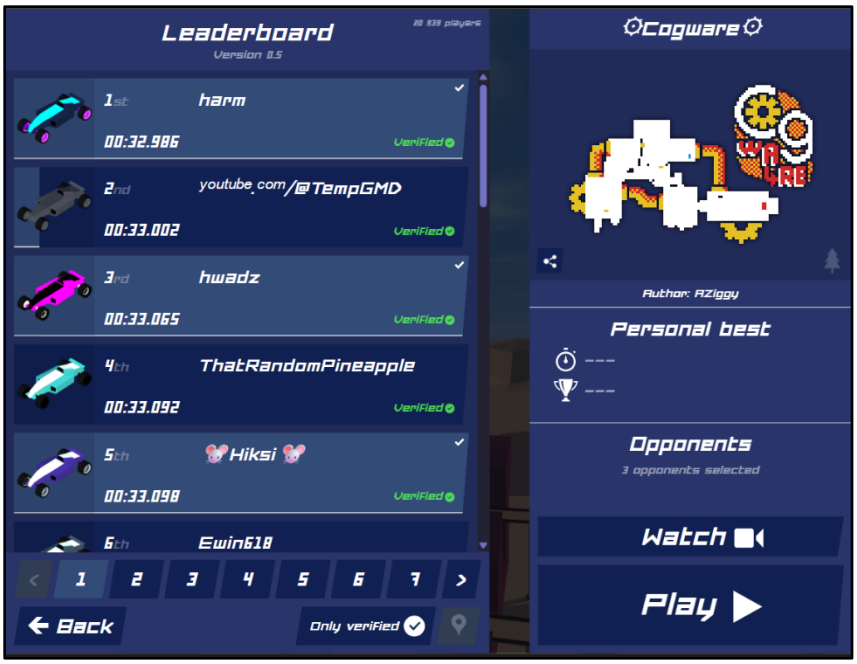


Figure 6 – the track details page in *PolyTrack*, showing all race attempts and their time
Three entries are selected, which can then be watched or played against.

Vector Racer and *PolyTrack* both have paginated time leaderboards for each track, portrayed in **Figure 6**. This contributes to a competitive atmosphere by giving players benchmarks to work towards, making it a fun element to include in the project (Lazzaro, 2009). However, the games' leaderboards are lengthy and lack search capabilities. It would be helpful to include query options for faster, more specific searches rather than navigating through each page.

2.3.2. Secondary Objective 2

"Enable players to race against AI racers of varying difficulty"

Vector Racer features AI at varying levels of difficulty that can be fun to play against, providing a range of challenges (Lazzaro, 2009).

1.	player 4	30.029		(#14)	(level 3 ai)
2.	player 1	30.171		(#18)	(level 3 ai)
3.	player 2	30.171		(#18)	(level 2 ai)
4.	player 3	30.171		(#18)	(level 1 ai)
5.	player 5	30.171		(#18)	(level 2 ai)
6.	player 6	38.029	(+3)	(#150)	(level 1 ai)

Track record: 28.533 all scores »

Figure 7 – the results of six AIs in *Vector Racer* after a session.
High-level AIs tend to perform better than low-level ones.

These difficulties are defined as levels, listed in **Figure 7**. However, players cannot specify their difficulty. This would be good to include to help players tailor a race that feels manageable and offers a fairer challenge (Lazzaro, 2009; Hunicke & Chapman, 2004). It is unclear exactly what determines AI levels in Vector Racer. In

any case, it is important to present them in a way that players can understand their effects and impact on gameplay so that they can adjust the levels accordingly.

2.3.3. Secondary Objective 3

"Enable players to personalise their gameplay experience"

Settings allow players to adjust game elements to what suits them. *PolyTrack* has such options, but most will likely not apply to the project, such as key bind controls for vector selection, as only the mouse would need to be used. *PolyTrack* also has a personalisation section that lets players change the colours of their race car. This makes the game more fun and provides a sense of individualisation, allowing players to race in their own style (Lazzaro, 2009). However, the shape of the car itself is the same for all players and cannot be changed, which may be to ensure fair gameplay. Since the *Racetrack* game represents racers as points, this constraint does not necessarily apply, so it would be interesting and potentially engaging to include this customisation (Lazzaro, 2009).

2.4. Other Games

As touched on in **Section 2.1**, other games were also researched. A notable one is *GPixel* (JesterDay Studio, 2026), which has similar mechanics to the *Racetrack* game but uses speed rather than velocity and an eight-spoked steering wheel for direction. It represents the car's speed as a gearbox, with each gear corresponding to a speed value. This could be a useful visual for players to assess their overall driving speed. *GPixel* also includes weather and track conditions that alter visibility and car traction, which could be an interesting feature to incorporate for a wide range of track experiences and challenges (Lazzaro, 2009).

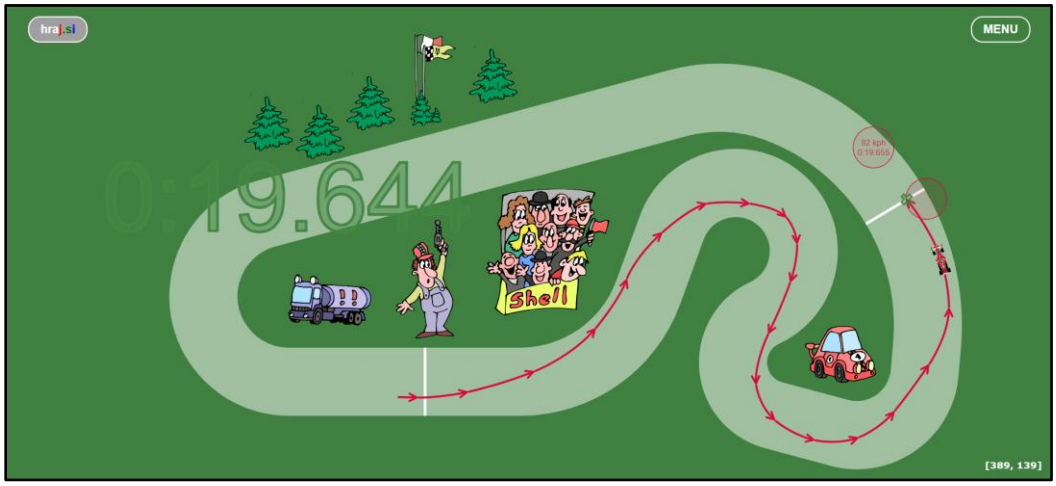


Figure 8 – a race from *Formulky*, demonstrating the Bezier curve movements around the track. The current time is also displayed when a white marker is passed.

Another researched game was *Formulky* (Formulky, n.d.). It is like the *Racetrack* game but uses a non-discrete field, and players drive along Bezier curves, as shown in **Figure 8**. While the *Racetrack* concept uses a discrete grid, including cars or the track itself shaped in a curved path would be interesting to explore. It is important to consider how this might work with collisions.

3. Chosen Technologies

At a high level, the application can be split into three components, corresponding to the areas of full-stack development (Zammetti, 2022):

- **Front End:** handle player interactions, such as UI and racing controls.
- **Back End**: handle server-side processes, like CRUD operations and multiplayer routing.
- **Databases:** persistently store racing or player data, primarily built tracks, race times, and customisations.

Collectively, these areas enable the original objectives to be fulfilled by providing a full channel of interaction. For example, when building a track, players can use the front-end interface to construct it, then submit it to the server for saving. A client-server architecture facilitates separation between the player and the server, enabling a barrier between client and server resources.

After researching several full-stack technologies (Zammetti, 2022; Kader & Ahmed, 2025), the following were settled on: HTML, CSS, and JavaScript for the front end; Node.js and Firebase for the back end; and MongoDB for long-term data storage. Vitest is a JavaScript framework which was chosen to construct unit and integration tests to ensure the application's validity (Node Package Manager, 2026-a).

Standard HTML and CSS provide a way to create a front-end interface with defined tags and styling. Other programming languages can be used to create interfaces, such as Java's JavaFX (OpenJFX, n.d.) or Python's Tkinter libraries (Python Software Foundation, 2026). However, HTML and JavaScript are easier to manage because HTML constructs the interface while JavaScript focuses on functionality, resulting in a clear separation (Zammetti, 2022). Similarly, CSS offers a way to swap styles through its style definition system, allowing elements to take on classes to achieve a particular appearance. JavaFX and Tkinter both support rendering HTML, but at that point, it is worth using HTML directly in a browser due to its accessibility, integration with JavaScript, and styling capabilities.

As for JavaScript, several libraries, such as Phaser (Phaser Studio, 2026), Kaboom (Kaboom, n.d.), and p5.js (p5.js, 2026), are suitable for web games and rendering, using a loop that updates the game per frame. However, since the *Racetrack* game is turn-based and more dependent on player-driven events than on continuous changes, standalone JavaScript was chosen. HTML also provides a means to draw to the screen with a canvas element. As the concept itself is simple, HTML and JavaScript together seem sufficient to implement gameplay aligned with the objectives, especially given JavaScript's extensive library ecosystem.

Following on from selecting JavaScript for the front end, it seems suitable to use Node.js for the back end. Using JavaScript throughout the application makes the development process easier than with other server-side languages like PHP, since it is used consistently (Iyer, 2013). Equally, it saves time spent learning another language (Heričko, 2017). While Node.js is less mature than PHP, it seems equally capable of achieving the objectives through JavaScript libraries, making it a viable alternative for the project's needs. The Node Package Manager (Node Package Manager, 2026-b) also comprises several packages that make the development process much smoother and more well-supported (Heričko, 2017).

MongoDB is used to store long-term data and is hosted on MongoDB Atlas (MongoDB, 2026). MySQL is suitable for storing sensitive information, such as login details (Šušter *et al.*, 2025). However, as the project grows, there may be several instances where fields are added or removed. MongoDB uses NoSQL, whose "schema-less" document-based nature can better accommodate these changes, making it more suitable for forward and backwards compatibility (Wu *et al.*, 2015). Additionally, some data, such as tracks and customisations, may be more appropriate and easier to capture in documents. MongoDB Atlas is a free Database-as-a-Service platform which has been chosen to host MongoDB, as it provides a GUI to create, view, and manage documents, which will be useful for testing and validation (MongoDB, 2026).

Firebase is a Backend-as-a-Service platform that was chosen for its extensive toolset and relevance to the project's needs (Firebase, 2026). One of its technologies is its real-time databases (RTDBs), which store data in real time and synchronise it with clients (Ohvyer *et al.*, 2019). This would be useful for online gameplay where states need to be propagated to each client, for example. Firebase also provides authentication methods for players to log in using an email address or a social media account (Firebase, 2026). This makes it easier to associate settings, customisations, and other relevant data with unique credentials that can serve as IDs for database logic. Using an existing trusted service is also more reliable, having been developed by a larger team and thoroughly tested, and much more secure than a system built in-house. Moreover, Firebase enables hosting full-stack applications through its Hosting features (Firebase, 2026). All these qualities make it a strong candidate for the project, which is why it was chosen. Finally, all tools which are used by the application are free under the Blaze plan, making it ideal for a simple project like this (Firebase, 2026).

Game development platforms were another consideration, as they, too, often enable full-stack development with the appropriate integrated technologies. Common examples include Unity and Unreal Engine, which are powerful and well-suited for complex games and include several libraries to cover all bases (Hussain *et al.*, 2020). However, most of its features will likely not be used to their full potential for the project, most notably its rendering and modelling capabilities, due to the simplicity of the *Racetrack* concept (Gardner, 1973; Hussain *et al.*, 2020). Moreover, there may be unnecessary overhead during setup. From previous experience, it has been quite cumbersome compared to web technologies, particularly in its script and component structure, which took a lot of time to learn. Overall, it is the versatility, ease, and abundance of documentation of web tools that make them desirable development tools for the project.

4. Application Requirements

Based on the researched game and available tools, a list of requirements can be collated to capture the desired features and improvements to be included in the project's implementation.

4.1. Requirements Glossary

The following glossary explains terms used in the requirements.

Term	Definition
AI Racer	A non-human racer.
Checkpoints	Points on the track that mark where players should go.
Decorations	Ornamental track elements which have no effect and are solely visual.
Ghost	A representation of a player's personal best time and its path.
Leaderboard	A list of player track completion times for a track.
Lobby	A room where players can join online to begin a race together.
Local Multiplayer	Several players racing together on the same device.
Materials	Traversable track elements that affect racer movements for the current turn if on it.
Obstacles	Track elements that cause players to crash when driven into.
Online Multiplayer	Several players racing together on separate devices.
Race	An instance of a session where a set of players race around a track.

Race Car	The vehicle a player uses to race around a track, which may or may not be a “car”.
Race Details	A set of live gameplay information: player velocities, turn count, player paths, and player checkpoint passes.
Race Options	A set of settings to apply to a race: player count, AI included, AI count, player race car types (for local sessions), and AI race car types.
Starting Line	The line on the track where all players start (and end) the race.
Track	A track which players can race around.
Track Details	The details of a track: name, description, and author.
Velocity	A racer’s current vector offset per turn.

4.2. Requirements List

Before officially beginning development, a collection of user stories was created in an Azure DevOps backlog for the project (AbdElazim *et al.*, 2020; Mogra, 2026; Williams, 2010). **Table 1** below lists these requirements, where orange, purple, and plain rows represent “Epics”, “Features”, and “User Stories” respectively (Williams, 2010). Each row includes the following details:

- **ID:** the unique ID of the user story in the DevOps project with a corresponding link.
- **Objective:** the objective the user story relates to (where “P” is primary, and “S” is secondary)
- **Title:** the concept the user story covers, including MoSCoW phrasing (Kravchenko *et al.*, 2022).

Note that the MoSCoW wording decides whether including a requirement is needed for the project’s success, not its corresponding objective.

ID	Objective	Title
351		Version 0 (MVP for Core Gameplay)
458		[V0] - Core
458	P4.1	[V0] - The player MUST be able to start the "Game"
361		[V0] - "Track" Viewing & Selection
431	P4.1	[V0] - The system MUST show all available "Tracks"
432	P4.1	[V0] - The player MUST be able to choose a "Track" to play
436	P4.1	[V0] - The system SHOULD show "Track Details"
365		[V0] - "Track" Representation
434	P4.1	[V0] - The system MUST be able to represent "Tracks"
377		[V0] - "Race" Gameplay
483	P4.1	[V0] - The system MUST show the "Track" the player chose to play
476	P4.1	[V0] - The system MUST show the player on the "Track"
433	P4.1	[V0] - The player MUST start a race from the "Starting Line"
447	P4.1	[V0] - The player MUST be able to control their "Race Car" when racing
487	P4.1	[V0] - The player MUST be bound to the "Track" confines when racing
448	P4.1	[V0] - The player MUST be able to complete an ongoing "Race"
497	P4.1	[V0] - The system SHOULD show the "Race Results" when completing a "Race"

ID	Objective	Title
372		[V0] - "App Settings"
435	S3	[V0] - The system MUST show all current "App Settings"
341		Version 1 (Building, Extended Gameplay, and Basic Saving)
364		[V1] - "Track" Building - Controls
439	P4.2	[V1] - The player MUST be able to save the "Track" they're building
586	P4.2	[V1] - The player SHOULD be able to undo building actions
587	P4.2	[V1] - The player SHOULD be able to redo building actions
446	P4.2	[V1] - The player SHOULD be able to set the "Track Details" of the "Track" they're building
588	P4.2	[V1] - The player COULD be able to test their "Track" while building it
522		[V1] - "Track" Building - Path
513	P4.2	[V1] - The player MUST be able to build their own "Track"
444	P4.2	[V1] - The player MUST be able to adjust the path of the "Track" they're building
538	P4.2	[V1] - The player COULD be able to define "Checkpoints" for the "Track"
523		[V1] - "Track" Building - "Materials"
521	P4.2	[V1] - The system SHOULD show all available "Materials" when building a "Track"
511	P4.2	[V1] - The player SHOULD be able to place "Materials" on the "Track" they're building
454	P4.2	[V1] - The player SHOULD be able to adjust placed "Materials" on a "Track" they're building
515	P4.2	[V1] - The player SHOULD be able to remove placed "Materials" from a track they're building
525		[V1] - "Track" Building - "Obstacles"
526	P4.2	[V1] - The system SHOULD show all available "Obstacles" when building a "Track"
527	P4.2	[V1] - The player SHOULD be able to place "Obstacles" on the "Track" they're building
528	P4.2	[V1] - The player SHOULD be able to adjust placed "Obstacles" on the "Track" they're building
529	P4.2	[V1] - The player SHOULD be able to remove placed "Obstacles" from the "Track" they're building
524		[V1] - "Track" Building - "Decorations"
520	P4.2	[V1] - The system COULD show all available "Decorations" when building a "Track"
437	P4.2	[V1] - The player COULD be able to place "Decorations" on the "Track" they're building
452	P4.2	[V1] - The player COULD be able to adjust placed "Decorations" on the "Track" they're building
451	P4.2	[V1] - The player COULD be able to remove placed "Decoration" from the "Track" they're building
411		[V1] - "Track" Management
559	P4.3	[V1] - The system SHOULD show the "Track Details" of the "Track" they've built
440	P4.3	[V1] - The player SHOULD be able to edit the "Track Details" of "Tracks" they've built
441	P4.3	[V1] - The player SHOULD be able to delete "Tracks" they've built
419	P4.3	[V1] - The player SHOULD be able to save copies of "Tracks" they've built
422	P4.3	[V1] - The player COULD be able to set "Tags" for "Tracks" they've built

ID	Objective	Title
344		[V1] - "Track" Viewing & Selection
445	P4.1	[V1] - The system MUST show all a player's saved built "Tracks"
450	P4.1	[V1] - The player COULD be able to filter all available "Tracks" by keywords
512	P4.1	[V1] - The player COULD be able to filter all available "Tracks" by their own
405	P4.1	[V1] - The system COULD show a preview of a "Track"
535		[V1] - Extended "Race" Gameplay
534	P4.1	[V1] - The system SHOULD affect players driving over "Materials"
536	P4.1	[V1] - The system SHOULD crash players driving into "Obstacles"
565	P4.1	[V1] - The player SHOULD be able to choose whether to continue a "Race" after crashing
386		[V1] - "Race Car" Selection
516	P4.1	[V1] - The system SHOULD show all available "Race Cars"
517	P4.1	[V1] - The player SHOULD be able to choose a "Race Car" to race with
518	P4.1	[V1] - The player SHOULD start a "Race" with their selected "Race Car"
572	P4.1	[V1] - The system SHOULD constrain player movement according to "Race Car" type
425		[V1] - Live "Race Details"
537	P4.1	[V1] - The system COULD show player paths throughout a "Race"
453	P4.1	[V1] - The system COULD show the turn count during "Races"
426	P4.1	[V1] - The system COULD show the player's current "Velocity"
427	P4.1	[V1] - The system COULD show "Checkpoints" the player has passed
332		Version 2 ("AI Racers", and "Local Multiplayer")
375		[V2] - "Race Options"
560	P4.1	[V2] - The system SHOULD show "Race Options" when selecting a "Track"
561	P4.1	[V2] - The player SHOULD be able to set "Race Options"
346		[V2] - "AI Racers"
568	P4.4	[V1] - The system SHOULD show "AI Racers" on the "Track"
562	P4.4	[V2] - The system SHOULD make "AI Racers" take optimal moves in "Races"
563	P4.4	[V2] - The system SHOULD apply "Material" effects to "AI Racers"
564	P4.4	[V2] - The system SHOULD apply "Obstacle" crashes when "AI Racers" drive into them
566		[V2] - "Local Multiplayer"
569	P4.5	[V2] - The player SHOULD be able to race with any number of "Local Players"
571	P4.5	[V2] - The player SHOULD be able to set the "Race Car" types of "Local Players"
567	P4.5	[V2] - The system SHOULD show the player and "Local Players" as distinct "Race Cars" on the "Track"
589		[V2] - "Ghost" Racing
590	P4.5	[V2] - The player COULD be able to race against their personal best for a "Track"
366		Version 3 ("Online Multiplayer", "Race Car Customisation", and Extended Data Management)
573		[V3] - "Track" Sharing

ID	Objective	Title
574	P4.1	[V3] - The system COULD show "Tracks" built by other players
575	P4.1	[V3] - The player COULD be able to play other players' "Tracks"
591		[V3] - "Track Leaderboards"
592	S1	[V3] - The system COULD save the best player times for a "Track"
593	S1	[V3] - The system COULD show all player times for a "Track"
396		[V3] - "Lobby" Management
576	S1	[V3] - The player COULD be able to create "Lobbies"
577	S1	[V3] - The system COULD show all available "Lobbies"
578	S1	[V3] - The player COULD be able to join an available "Lobby"
580	S1	[V3] - The system COULD show players present in a "Lobby"
579	S1	[V3] - The player COULD be able to leave a "Lobby"
581	S1	[V3] - The player COULD start a "Race" from a "Lobby"
582		[V3] - "Online Multiplayer"
585	S1	[V3] - The player COULD be able to race against other players online
397		[V3] - "Race Car Customisation"
583	S3	[V3] - The system COULD show the player's current "Race Car" appearance
584	S3	[V3] - The player COULD be able to edit their "Race Car" appearance

Table 1 *– the list of requirements outlined for the project based on the researched games and technologies.*

5. Design and Architecture

This section covers the design and implementation of the application's features. It will hopefully inform the application's structure before referring to the resources when explaining each feature in line with the objectives.

5.1. Files and Execution

As mentioned in **Section 2**, the application is built using basic web technologies: HTML, JavaScript, and CSS. Firebase offers a Hosting service for deploying an application through its free-to-use Blaze plan. However, this supports only static web pages, which do not accommodate the application's dynamic nature, including live updates and routing. Firebase's Cloud Functions resolve this, enabling Node.js to route the client accordingly or respond to data requests from them (Moroney, 2017). This is used, for example, to load the tracks list with all tracks or to receive player input during races.

A basic Firebase folder structure can be created by executing `firebase init` in the current directory or repository to produce the required files. This asks a series of questions, including whether Node.js will be used for the application.

Directory	Handled By	Purpose
`/functions`	Firebase	Dedicated to storing functions, which must contain an `index.js` file that exposes them.

`/models`	Mongoose (MongoDB)	The schemas used by Mongoose to perform CRUD operations on MongoDB data.
`/services/firebase.admin`	Firebase	The credentials to provide the server access to invoke Firebase operations.
`/src`	Node.js	Code which is not client-facing. These are scripts concerned with game logic, namely classes, enumerations, and utilities.
`/views`	Node.js	All EJS views (essentially HTML pages) for the application. These are then provisioned by Node.js when routing to them.
`/app.js`	Firebase	The main application that routes all requests and processes game logic.
`/firebase-service-account.json`	Firebase	Firebase credentials that are read by `/services/firebase.admin` to authenticate the server.
`/index.js`	Firebase	A bucket file exposing all relevant functions associated with the current directory.
`/package.json`	Node.js	Details about the application, including the main script to execute when running the application and the dependencies to install.
`/public`	Node.js	Static files that are client-facing, particularly client-side JavaScript, CSS, HTML web pages, and other similar resources.
`/tests`	Vitest	A collection of tests to ensure the application functions accordingly.
`.firebaserc`	Firebase	The name of the Firebase project where Firebase resources, such as RTDB, are found.
`/firebase.json`	Firebase	Describes which resources are functions, as well as routing information when the application is deployed to Firebase.

***Table 2** – the file structure of the application, including the resource, which component handles it, and its purpose.*

Table 2 lists the core files and folders created for the entire application. For each file or folder, it lists the directory, the application component primarily responsible for it, and its purpose. Some folders have been summarised for brevity and will be discussed in more detail later. Other superfluous files which do not directly contribute to the application, such as `.gitignore`, have been omitted. Throughout the report, where directories are mentioned, assume the root of files or folders is `Application`.

Firebase's Cloud Functions service looks for functions with `index.js` files to execute, as specified in the configuration file `firebase.json` (Moroney, 2017). In turn, the index.js file exports and exposes the Express portion of the application, `functions/app.js`. The configuration file also defines routing behaviour.

```
// functions/app.js

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
app.set("views", path.join(__dirname, "views"));
app.set("view engine", "ejs");

app.use(cors());
```

```
app.use(express.json());
app.use(express.static(path.join(__dirname, "../public")));
app.use("/core", express.static(path.join(__dirname, "/src/core"))); // core classes etc.

const { version } = pkg;
app.locals.version = version;

// ... code and routes ...

export const appExpress = app;  // firebase
export default app;             // basic node
```

***Code Sample 1** – the core setup for app.js to prepare routing and resources.*

```
// functions/index.js

import { onRequest } from "firebase-functions/v2/https";
import { appExpress } from "./app.js";

export const app = onRequest(appExpress);
```

***Code Sample 2** – the core setup for app.js to prepare routing and resources.*

Code Sample 1 shows the core part of `app.js`, which sets up routing and exports the Express application; index.js in Code Sample 2 then exports it to expose it to Firebase. The application defines the view engine which is being used and where to find those views. It also uses the public folder by default, so that the views, despite being kept under `functions/views`, can use CSS stylings and other scripts stored in `public`. Likewise, the application defines an additions/core directory that front-end scripts in public can use from `src/core`. This enables the front-end to access exactly the resources it requires and doubles as a single point of reference should the folders move, making development easier.

5.2. Pages and Client Scripts

The application has several, each represented by an Embedded JavaScript file (EJS). EJS operates the same way as HTML but also allows data to be dynamically embedded in the page. This can be raw data or reusable HTML elements called “partials”.

Page	View (`functions/views`)	Scripts (`public/js`)	Description
Home	`home.ejs`	`page-home.js`	An introduction to the game and an option to sign in.
Tracks List	`tracks.ejs`	`page-tracks.js`	All available tracks recognised by the game.
Track Builder	`builder.ejs`	`page-builder.js`	The track builder which the player can use to construct their own track.
Lobbies	`lobbies.ejs`	`page-lobbies.js`	All available lobbies to join.
Lobby	`lobby.ejs`	`page-lobby.js`	Details associated with the joined lobby, as well as customisation options.
Race	`race.ejs`	`page-race.js`	The race the player is a part of, allow them to take their turns and play the race.

404 Error	`error-404.ejs`	-	Default page when attempting to route to an endpoint which does not exist.

***Table 3** – all EJS views, their corresponding scripts, and their purpose.*

Component	Partial (`functions/views/partials`)	Description
Page Head	`head.ejs`	Information about the page, such as the title, and common resources used, such as styling and icons.
Navigation Bar	`nav.ejs`	A header for the site with links to other pages.
Toast Notification	`toast.ejs`	A template to display notifications to the player.
Footer	`footer.ejs`	A footer with additional information, such as the current account and game version.

***Table 4** – all EJS partials and their purpose.*

Table 3 lists the available pages, their corresponding views and scripts, and a description of their functionality. Additionally, **Table 4** lists all partials and their purposes. Partials are standalone HTML fragments, so they do not have corresponding scripts.

```
<!-- functions/views/lobby.ejs -->
<!DOCTYPE html>

<html lang="en">
<head>
    <%- include("./partials/head.ejs") %>
    <script type="module" src="/js/page-lobby.js"></script>
</head>
<body>
    <div class="container">
        <nav>
            <%- include("./partials/nav.ejs") %>
        </nav>

        <!-- content -->

        <%- include("./partials/toast.ejs") %>

        <footer>
            <%- include("./partials/footer.ejs") %>

            <-- example – footer.ejs evaluates to:
                <span id="sign-in-account">Logged in as: ...</span>
                <br>Created by Muhammad Mogra
                <br>Version <%= version %>
            -->
        </footer>
    </div>

    <script>
        window.lobby_ = <%- JSON.stringify(lobby) %>;
```

```
    </script>
<body>
<html>
```

Code Sample 3 *– the `lobby.ejs` page content with four partial tags, along with embedded data for `page-lobby.js` to use.*

Partials allow a single source to modify content across all pages, such as the navigation bar at the top of the page, ensuring consistent designs. EJS syntax can then be used to render this on the page with `<%- include(...) %>`, or to pass data with `<%- ... %>`, as shown in **Code Sample 3** for the lobbies page.

5.3. Component Integration and Routing

As mentioned in previous sections, the front-end interface is developed with HTML, CSS and JavaScript, while the back-end databases are MongoDB and Firebase.

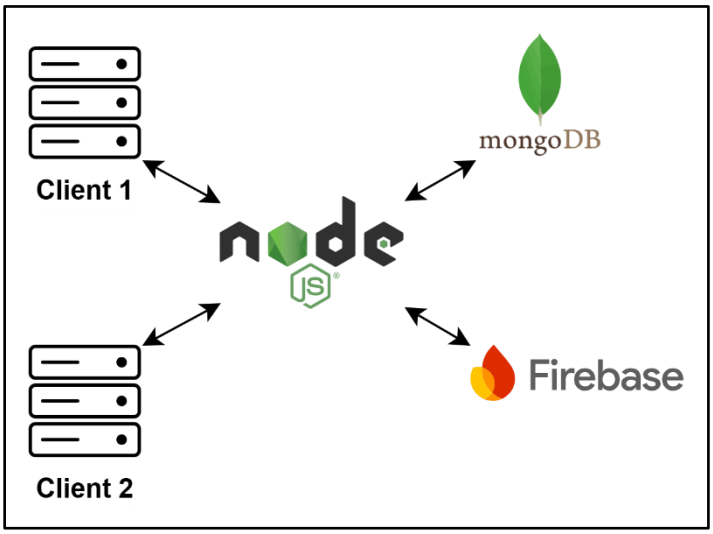


Figure 9 *– the integration between the front-end, back-end, and datastores for the application.*

Node.js is the middleware that connects these components by routing client requests and data, as illustrated in **Figure 9**. It includes operations which allow HTTP requests to be made to the client. These requests can be used to deliver content or data to and from the client, such as the track list and individual track data. However, the base HTTP package for this is heavy. Express, an NPM package, has been used to make setting up these routes much less verbose (Ugwumba, 2025).

```
// functions/app.js

app.get("/api/tracks/:trackId", async (request, response) => {
    try {
        // ... validation code & alternative routes

        response.status(200).json({ success: true, track });

    } catch (error) {
        console.log(error);
        response.status(500).json({ success: false, error });
```

```
    }
});

app.get("/tracks", async (request, response) => {
    try {
        let tracks = await getTracks();
        let leaderboards = await getLeaderboards();
        response.render("tracks", { title: "Tracks List", tracks, leaderboards });

    } catch (error) {
        console.log(error);
        response.status(500).json({
            success: false, error: `Failed to load tracks: ${error}` });
    }
});
```

***Code Sample 4** – the endpoints for `/api/tracks` and `/tracks`.*

For example, **Code Sample 4** demonstrates the endpoint that returns the Tracks page view (`track.ejs`), as well as the endpoint which returns the data for a particular track by ID. Endpoints have been split based on whether they deliver page content or data, as indicated by the presence of `/api` at the start of the path. This helps distinguish between endpoints which would otherwise have the same name. For example, `/tracks` shows the track page with all tracks listed, while `/api/tracks` returns all track data. Furthermore, **Code Sample 4** demonstrates how data is passed to pages using the EJS view engine, which provides it as a dictionary, making it easy to read and display as content.

Express works by checking each endpoint in order of its script definition. In **Code Sample 4**, if the player were to request `/tracks`, Express would first check whether `/api/tracks` matches, then match it to`/tracks`. However, there may be cases where the player requests an endpoint that does not exist, resulting in no content being served and leaving them locked on a default page.

```
// functions/app.js

app.use((request, response) => {
    if (request.path.startsWith("/api")) {
        return response.status(404).json({
            success: false, error: `"${request.path}" is not an endpoint`});
    }

    response.render("error-404", { title: "404 Not Found" });
});
```

***Code Sample 5** – the failsafe endpoint catches any request to unrecognised endpoints.*

To make the experience smoother, a default endpoint has been added at the end of the script, as shown in **Code Sample 5**. If the user attempts to visit a page that does not exist, they are presented with a dedicated 404 page that informs them and provides links to navigate elsewhere. Alternatively, if they make an API request, they will receive a 404 response.

Given error handling, a common pattern in endpoint invocations, particularly when creating new resources such as a lobby or race, is to operate in two parts. For example, when creating a lobby, the host first sends a `POST /api/lobbies` request. The server creates a new lobby with an ID, which the host's front-end script uses to navigate to the Lobby page for that lobby. Since the endpoints have been separated by those which return a page and those which return data (i.e., `/` and `/api`), this two-part process makes it simpler to return the response of malformed requests to the player with clear formats to handle accordingly, such as displaying a notification on the client's screen if the request could not be processed.

5.4. Data Structures and Serialisation

5.4.1. Objects and Class Diagrams

As mentioned previously, an OOP approach was chosen when developing the project due to its benefits in conciseness and reusability.

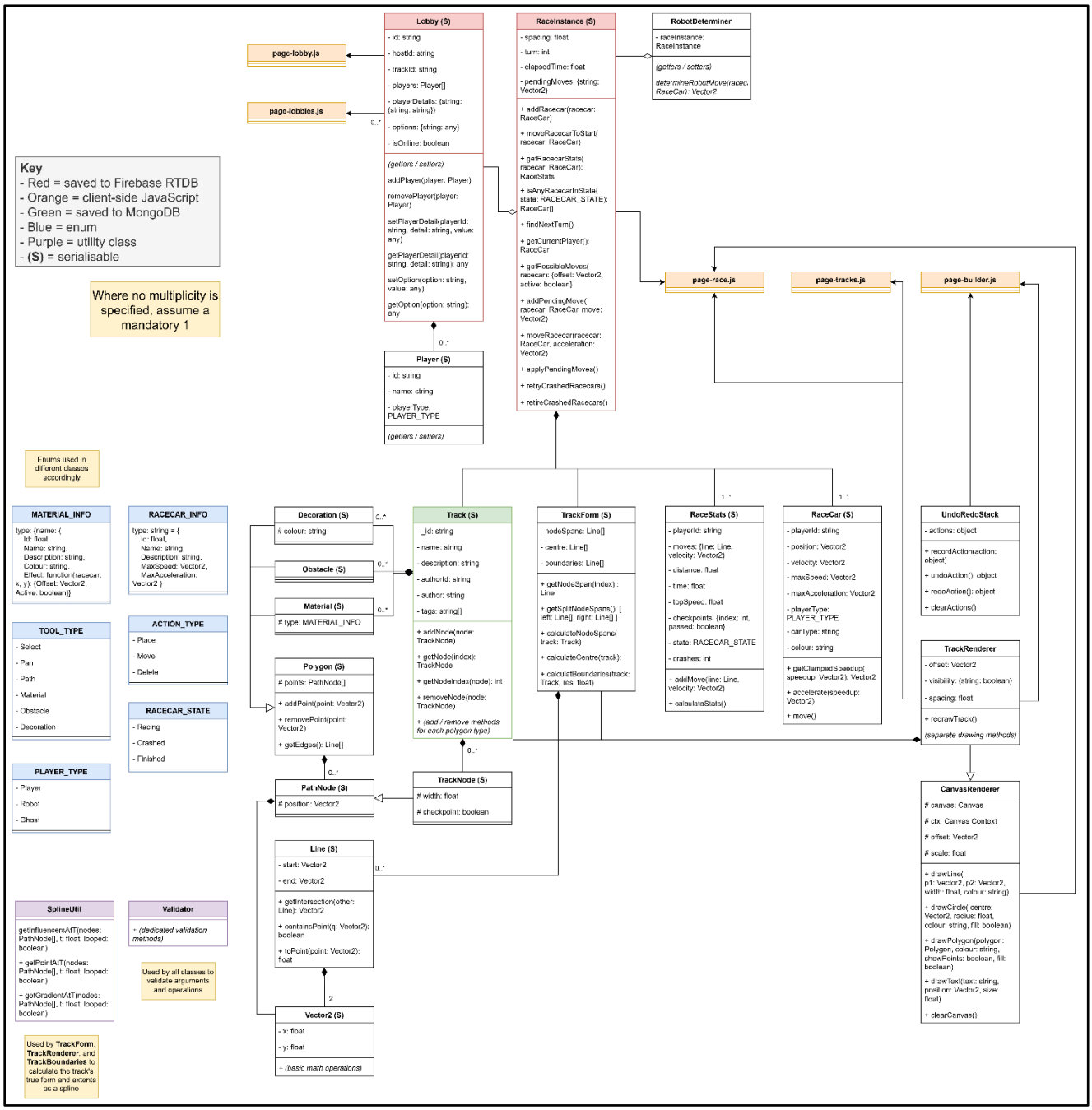


***Figure 10** – a class diagram comprising all resources in `functions/src`. Note the key in the top left.*

Figure 10 shows the holistic class diagram of all classes, enumerations, and utilities in `functions/src`, and **Table 5** lists the purpose of each. Classes that are saved to MongoDB and Firebase include custom serialisation methods to convert instances to and from objects when they are saved to and retrieved from their respective datastores. Each relevant class has a static `toInstance()` and non-static `toObject()` method, taking an object and converting it to the corresponding instance or converting the instance to an object, respectively. Finally, the built-in `JSON.stringify()` method is commonly used throughout the application to serialise data objects in JSON format. The method can directly take a JavaScript object and convert it, or invoke the `toJSON()` method of the given class instance. This feature has been implemented by adding a corresponding toJSON() method to each serializable class, which returns the result of `toObject()` invoked on the class itself.

```
// functions/src/core/classes/geometry/vector2.js

toObject() {
    return { x: this.#x, y: this.#y };
}

static toInstance(object) {
    return new Vector2(object.x, object.y);
}

toJSON() {
    return this.toObject();
}
```

***Code Sample 6** – the serialisation methods in the `Vector2` class.*

Code Sample 6 demonstrates these serialisation methods using the `Vector2` class as an example. Each class constructor includes validation by the `Validator` utility to catch malformed data. Classes that are aggregates of saved classes also have their own serialisation methods for conversion as required. As for `Track` and `Lobby`, all their details are saved to their respective databases, as all information in their objects must be propagated to players. The same is true for `RaceInstance` objects, except form, represented by a `TrackForm` object, is not stored. This is due to the density of its data, discussed in later sections. Instead, it is reconstructed on the receiving client as needed.

5.4.2. MongoDB Data

A core objective of the project is for players to save and manage tracks they have built. This is made possible by MongoDB, which allows them to store their tracks long-term. Additionally, multiplayer features require that leaderboard scores be saved.

```
// functions/app.js

const uri = "mongodb+srv://...";

mongoose.connect(uri)
    .then((result) => {
        console.log("MongoDB Starting");
        mongodbReady = true;
    })
    .catch((error) => {
        console.log(error);
        console.log("[CONNECTING WITHOUT DATABASE!]");
```

```
    });
```

***Code Sample 7** – the connection to the MongoDB database, where `uri` is the connection string.*

Similar to Express, MongoDB has its own dedicated NPM package that provides a means to communicate with a MongoDB database. In the project's case, this was hosted on MongoDB Atlas. However, MongoDB syntax alone can be complex, with queries typically defined using BSON documents. This resembles JSON but can be difficult to construct and read. Instead, the Mongoose package has been used, which provides a much simpler way to access the database. **Code Sample 7** shows how this is set up in `app.js`.

```
// functions/models
import mongoose from "mongoose";

const Schema = mongoose.Schema;

const trackSchema = new Schema({
    name: { type: String, required: true },
    description: { type: String, required: true },
    authorId: { type: String, required: true }
}, { strict: false });

const Track = mongoose.model("Track", trackSchema);

export default Track;
```

***Code Sample 8** – the Mongoose-defined schema for a `Track` object.*

Templates can be prepared in the Node.js application using the expected data schema. The folder `function/models` holds the schemas for both tracks and leaderboards. **Code Sample 8** shows the schema for tracks, which requires a name, description, and author ID. The schema is kept non-strict to ensure that other track components are saved, namely, structural elements such as materials. Additionally, an ID is automatically generated for the track when added to the MongoDB database, corresponding to `_id`. This is also why `_id` in the `Track` class is named with an underscore, discussed later. The leaderboard schema follows a similar pattern, instead storing details of scores. This is covered in more detail in a later section.

(section continues)

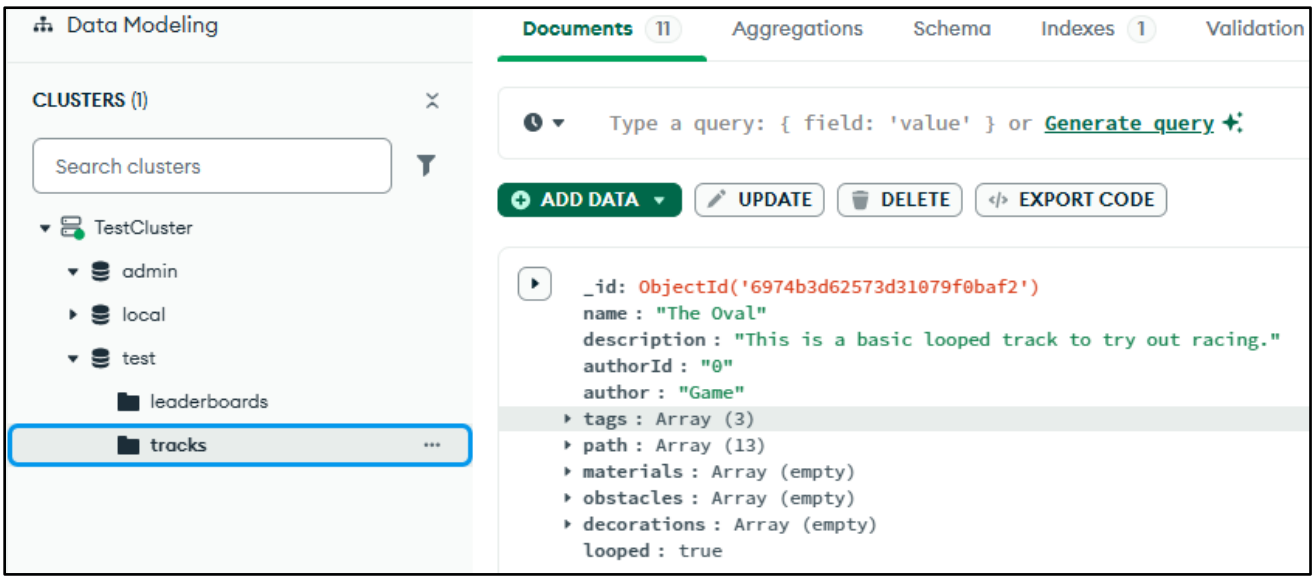


Figure 11 *– an example of the track "The Oval" stored in the MongoDB database.*

The resulting schema model can then be imported into other scripts to extract data, provided a connection to the MongoDB database has been established. All tracks are retrieved using `TrackModel.getTracks()`, which is used by the `/tracks` endpoint in **Code Sample 4**. Finally, **Figure 11** shows an example of track data in the MongoDB database, including all the required fields from **Code Sample 8**.

5.4.3. Firebase Realtime Database Data

The secondary objectives stipulate that the application enables players to race with others online. Without a network connection, the application only supports local multiplayer. Firebase RTDB helps with this by storing temporary data, namely lobby and race details. This is a better fit than MongoDB, which is better suited to long-term data. Connections can also be created on the client to listen for any changes to a specific race instance.

```
import { ref, onValue } from "firebase-database";

let lobbyRef = ref(db, `lobbies/${lobby.getId()}`);

onValue(lobbyRef, (snapshot) => {
    let lobbyObj = snapshot.val() || null;

    // redirect to race - if no race, will go back
    if (!lobbyObj) window.location.href = `/races/${lobby.getId()}`;

    lobby = Lobby.toInstance(lobbyObj)

    refreshLobbyDetails();

    if (!lobby.getPlayers()[playerId]) window.location.href = "/tracks";
});
```

Code Sample 9 *– the `onValue` connection in `page-lobby.js` that listens for any changes to the lobby and displays them.*

For example, **Code Sample 9** shows the `onValue` connection in `page-lobby.js`, a client-side script that listens for changes to the lobby reference in the RTDB. Any modifications to the data, such as player additions or detail adjustments, are propagated to the page. This is discussed in more detail in a later section.



***Figure 12** – the RTDB with an instance of a lobby and race.*

Figure 12 shows an example of the Firebase RTDB with a created lobby and race. Each one is automatically assigned a unique ID, which is provided to the corresponding `Lobby` or `RaceInstance` object.

5.5. User Login System

Firebase also includes a user authentication system that allows users to log in with their accounts. The only option available is to sign in with Google, available on the Home page. Users may also choose to play anonymously. The application has accounted for this by enabling features such as lobby creation and gameplay, but turning off others associated with saving, like track building and scores.

```
async function initAuth() {
    if (!authInitialised) {
        await setPersistence(auth, browserLocalPersistence);
        authInitialised = true;
    }

    onAuthStateChanged(auth, async (user) => {
        if (authResolved) return;

        if (user) {
            authResolved = true;

            authPromiseResolver(user);
            updateFooterAccount(user);

        } else if (!auth.currentUser) {
            const anon = await signInAnonymously(auth);
            authResolved = true;

            let user = anon.user;
            authPromiseResolver(user);
```

```
            let name = generateAnonymousName();
            await updateProfile(user, { displayName: name });

            updateFooterAccount(user);
        }
    });

    return authPromise;
}
```

***Code Sample 10** – authenticates the user with their current account or anonymously.*

Code Sample 10 shows the `initAuth()` commonly used throughout client-side scripts to authenticate users. This returns a user object with details about their account, such as user ID and display name, which can be used when displaying content associated with them. Users who do not have an account are signed in anonymously. As anonymous users do not have display names, they are generated as “Anonymous” followed by a random 6-digit number. These users still have user IDs for unique authentication, but this may change between sessions. For this reason, only signed-in users can save data, as anonymous users may never be able to access their data again, even if it is saved. The user’s updated display name is then shown at the bottom of each page so that they are aware of the account they have signed into.

5.6. Icons and Styling

As mentioned in **Section 3**, CSS was chosen for styling given its integration with HTML and JavaScript. Additionally, Bootstrap was selected as a styling framework, as it provides a suite of widgets and styles that support the application's design (Bootstrap, 2026). While other frameworks do exist, such as Tailwind (Tailwind CSS, 2026), Bootstrap was chosen due to familiarity with it from previous work. Additionally, icons were created using Lucide, a repository of open-source SVG images which can be customised (Lucide, 2026). Likewise, it was chosen for its reliability, freedom of customisation, and familiarity, despite Bootstrap's own icon system (Bootstrap, 2026).

6. Implementation and Algorithms

This section covers the application's functionality in more detail, focusing on page operations and the rationale behind key object structures.

6.1. Track Representation

This section explains how tracks are represented, the rationale behind this representation, and how track building works.

6.1.1. Track Objects

Among the games researched, there are two ways to create tracks: one using an image map, as in *Vector Racer*, and the other using interconnected parts, as in *PolyTrack*. Methods beyond these games mostly involve arranging track paths as a series of nodes. Altogether, these methods can be summarised as either sequences of nodes or 2D arrays, considering that images are essentially arrays of pixel colours. In the end, the node approach was settled on.

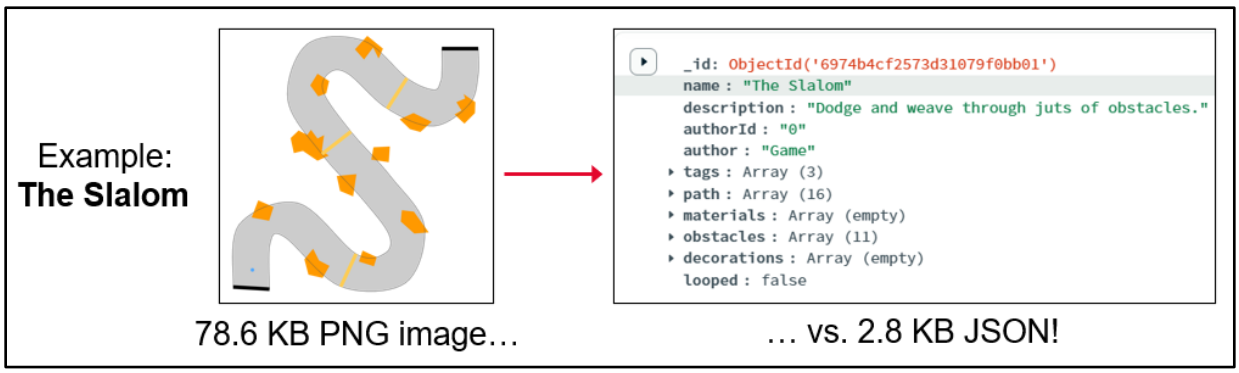


Figure 13 *– an example of the track "The Slalom".*
The right shows the node data used to construct the track.
The left takes the plain canvas output of the construction without adornments.

Using a series of nodes proved more memory-efficient than representing a track as a 2D array, as shown in **Figure 13** for the track “The Slalom”. This is because data is only required for the node positions, rather than a colour corresponding to each pixel, and it was the primary driver for pursuing this method.

Materials and obstacles in the researched games make the games more challenging and may be enjoyable to include to make them more interesting (Lazzaro, 2009). Images are effective at marking materials and obstacles, as the player can draw wherever they like. However, this is not as simple for a node-based approach. To work around this, materials and similar elements are represented as polygons composed of several points. These polygons are independent of the track and can be freely moved around.

In terms of placement, this is also simple for an image. Considering a 2D array, each cell could be painted and have a track or material indicator applied to it, and rendered accordingly; with nodes, this is more complicated. Like how tracks are represented with nodes, materials, and the like, polygons are isolated from the track. This allows players to create custom shapes, as in painting an image, to maintain freedom (Lazzaro, 2009). For the image method, there are also only so many colours before the player would find it difficult to tell them apart, particularly if they are close together. This polygon approach exceeds this by referring to elements by name and property rather than colour, which is more distinguishable.

Another element is detecting race cars on top of materials, which is necessary to affect player movement for the next turn. A 2D array can be indexed with the player’s position to determine whether they are on a material or obstacle. However, with nodes and polygons, calculations are required to determine track and material extents, which are then compared to the race car’s position. This can be achieved by using the ray-casting algorithm to determine whether the race car lies within a polygon (Hormann & Agathos, 2001). Similarly, calculations can be made to check whether the racecar’s route crossed a track boundary and decide whether it should crash thereafter (Antonio, 1992). These processes are detailed in later sections.

Despite the relatively complex workaround to address these limitations, it is beneficial to use the node approach over the 2D array for efficient storage, especially given the limits of what MongoDB and Firebase offer for free (MongoDB, 2026; Firebase, 2026). However, care must be taken to ensure a sufficient UX, with supportive tools for placing nodes and creating polygons easily (Nielsen, 1994). Additionally, selected points on the grid must be mapped to track materials to affect velocity or cause crashes accurately. This is presented when discussing the Track Builder in **Section 6.2**.

6.1.2. Spline Calculations and Track Form

As mentioned earlier, splines can be used to give the track a curve. The form of the track is the most important visual aspect, as it must be intuitive enough to allow the player to construct a track, while also producing a well-formed curve in a reasonable time.

Many methods for constructing splines exist, such as cubic curves, natural cubic curves, B-splines, and Bezier curves (Graphics in 5 Minutes 2022-a, 2022-b; Zheng, 2021). However, most of these approaches have drawbacks: they are either too computationally intensive to compute for even a small number of nodes, affect the entire spline when moved, or do not pass through the points. Catmull-Rom splines solve all these issues by constructing a spline that passes through all points, with the curvature at each point defined by a set of neighbouring nodes (Catmull & Rom, 1974). As such, an approach was chosen to give the node series a form.

```
static getPointAtT(nodes, t, looped) {
    let [p0, p1, p2, p3] = SplineUtil.getInfluencersAtT(nodes, t, looped);

    t %= 1;

    let tt = t * t;
    let ttt = tt * t;

    let q0 = -  ttt + 2*tt - t;
    let q1 =  3*ttt - 5*tt + 2;
    let q2 = -3*ttt + 4*tt + t;
    let q3 =    ttt -   tt    ;

    let pos0, pos1, pos2, pos3;

    // * TODO: change this for this & gradient
    if (nodes[p0] instanceof PathNode) {
        pos0 = nodes[p0].getPosition();
        pos1 = nodes[p1].getPosition();
        pos2 = nodes[p2].getPosition();
        pos3 = nodes[p3].getPosition();
    } else {
        pos0 = nodes[p0];
        pos1 = nodes[p1];
        pos2 = nodes[p2];
        pos3 = nodes[p3];
    }

    // halve: because t ranges from 0 to 2 with these equations
    let tx = 0.5 * ( (pos0.x * q0) + (pos1.x * q1) + (pos2.x * q2) + (pos3.x * q3) );
    let ty = 0.5 * ( (pos0.y * q0) + (pos1.y * q1) + (pos2.y * q2) + (pos3.y * q3) );

    return new Vector2(tx, ty);
}
```

Code Sample 11 *– the method to calculate the position of a progression t along a Catmull-Rom spline.*

Code Sample 11 shows the calculations of the Catmull-Rom spline carried out by the utility class `SplineUtil`. (Catmull & Rom, 1974; javidx, 2017), It takes the set of nodes corresponding to the track and a value t representing the progression along the track and whether the set of nodes is looped. It then takes the four closest neighbours to t, each assigned a cubic function that determines the node's influence on the point. For open-

ended tracks, these influences are limited and stacked at the edge nodes; likewise, if there are fewer than four nodes in a track. After calculating the influences, the final position of the point at t is calculated based on the positions of the influencing nodes and returned as a `Vector2` object.

```
// functions/src/classes/track/track-form.js

calculateBoundaries(track, res = 0.1) {
    // the actual collidable boundaries of the track (should correspond to spline)
    // * "res" = resolution - determines how "fine" the boundaries are

    Validator.isInstance(track, Track);
    Validator.isNumeric(res);

    this.#boundaries = [];

    let size = track.getSize();
    if (size == 0) return;

    let obstacles = track.getObstacles();
    let looped = track.isLooped();

    let left = [], right = [];

    for (let n = 0; n < track.getPath().length; n++) {
        let extents = this.getNodeSpan(n);
        left.push( new PathNode(extents.start) );
        right.push( new PathNode(extents.end) );
    }

    // track edges (same idea as drawing track path)
    let tmax = looped ? size : size - 1

    for (let t = 0; t < tmax; t += res) {
        let left1  = SplineUtil.getPointAtT(left , t      , looped);
        let left2  = SplineUtil.getPointAtT(left , t + res, looped);
        let right1 = SplineUtil.getPointAtT(right, t      , looped);
        let right2 = SplineUtil.getPointAtT(right, t + res, looped);

        this.#boundaries.push( new Line(left1, left2) );
        this.#boundaries.push( new Line(right1, right2) );
    }

    // start / finish line are boundaries for unlooped tracks
    if (!looped) {
        let start = new Line(left[0].getPosition(), right[0].getPosition());
        let end = new Line(left[size - 1].getPosition(), right[size - 1].getPosition());

        this.#boundaries.unshift(start);
        this.#boundaries.push(end);
    }

    // obstacles
    let obstacleEdges = obstacles.map(o => o.getEdges()).flat();
    let edgeLines = obstacleEdges.map(e => new Line(e[0], e[1]));
    this.#boundaries = this.#boundaries.concat(edgeLines);
}
```

***Code Sample 12** – the main method in `TrackForm` to calculate the boundaries, used to draw track forms.*

`TrackForm` is a class that calculates the boundaries of a track, producing an object with details used for both rendering and collision detection. To do this, it first determines the left and right edges by extending the player-placed central nodes along their gradients, retrieved via `getNodeSpans()`. Then, based on a resolution, it draws small lines at a specified resolution, which follow the curvature of the spline formed by each respective side at a progression t. The resolution describes the step in t, with a lower resolution value meaning greater precision in calculating the spline's curvature. If the track is open-ended, both the start and end are considered boundaries. **Code Sample 12** shows this procedure, with **Section 6.1.4** providing a visualisation of a track to illustrate the left and right sides better. Finally, obstacle edges are also gathered and added as boundaries. Collectively, these boundaries are also used for collision detection, which is discussed later.

6.1.3. Track Visualisation

All tracks are rendered using `TrackRender` objects, which take a `TrackForm` object and draw the track from its boundaries, along with other details such as checkpoints and materials. This is also the case for the tracks, which are visualised throughout the application, such as track entries in the tracks list and the race itself.

```
// functions/src/core/classes/rendering/TrackRenderer.js

constructor(track, canvas, spacing = 10) {
    super(canvas);

    this._visibility = {nodes: true, wires: true, grid: true,
        path: true, materials: true, obstacles: true, decorations: true};

    this.setTrack(track); // also sets TrackForm
    this.setSpacing(spacing);
}

redrawTrack() {
    this.clearCanvas();

    this._trackForm = new TrackForm(this._track);

    let visibility = this._visibility;

    let visuals = [ // corresponds to draw order
        { Method: this.renderPath,          Key: "path" },
        { Method: this.renderDecorations,   Key: "decorations" },
        { Method: this.renderMaterials,     Key: "materials" },
        { Method: this.renderObstacles,     Key: "obstacles" },
        { Method: this.renderCheckpoints,   Key: "path" },
        { Method: this.renderPathOutline,   Key: "path" },
        { Method: this.renderWires,         Key: "wires" },
        { Method: this.renderNodes,         Key: "nodes" },
        { Method: this.renderDirection,     Key: "path" },
        { Method: this.renderGridLines,     Key: "grid" }
    ]

    for (let entry of visuals) {
        Validator.keyPresence(visibility, entry.Key);
        if (visibility[entry.Key]) entry.Method.call(this);
    }
```

```
}
```

***Code Sample 13** – the constructor and rendering function of the `TrackRender` class.*

Code Sample 13 shows a code snippet from the `TrackRenderer`, consisting of its constructor and the `redrawTrack()` rendering function. A `TrackRenderer` object consists of a property called `_visibility`, a dictionary of string keys and boolean values, where each string represents a track element and its value indicates whether it is drawn to the canvas or not. For example, if `renderPath` is true, then the track path is drawn. This exposure and simplification enable other scripts to toggle them depending on the use case. For example, these options are all toggleable on the Track Builder page, whereas entries on the Tracks List page show only elements without nodes or adornments. The components are then drawn in a specified order to place them on top of one another.

6.2. Track Building

This section covers the features of the track builder. Based on the analysed games, only *PolyTrack* included a track builder. While Vector Racer allows players to upload a coloured image, the colour palette is very limited, and there is no way to test the game directly. Nevertheless, it does provide the player with creative freedom. *PolyTrack* resolves the issues of *Vector Racer* but pertains to a typical 3D racing game. A track builder that

provides a similar level of freedom is essential to the project's success (Primary Objective 4.2), making it an enjoyable experience and surpassing what existing games offer.

6.2.1. Track Builder Overview

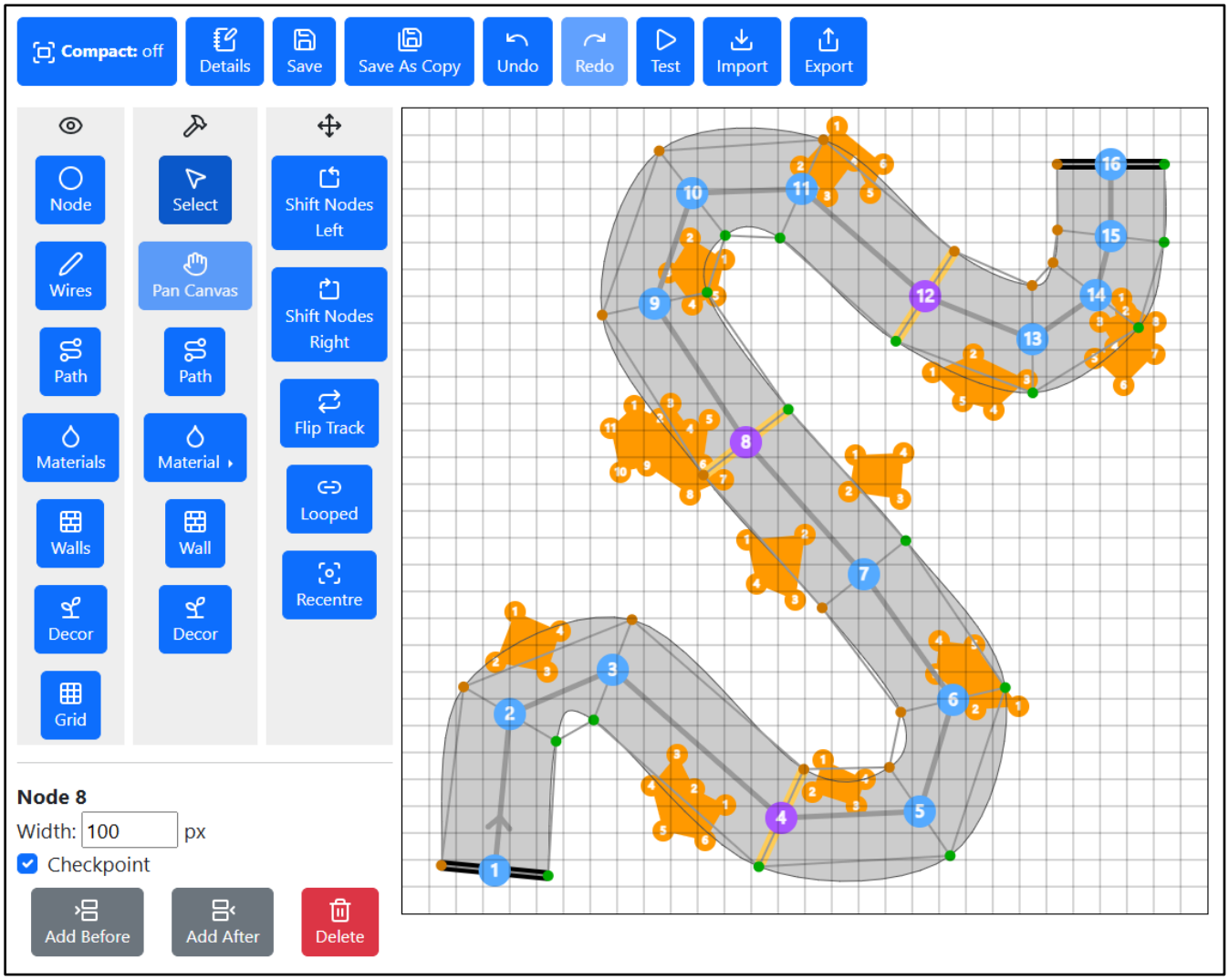


Figure 14 *– the track builder with the track "The Slalom" loaded in.*
Compact mode is enabled to show all button labels. Node 8 (centre) has been selected.

Figure 14 shows a holistic view of the track builder with the track “The Slalom” imported, an open-ended track with 16 nodes, 3 of which are checkpoints marked in purple, and 10 obstacles, coloured in orange.

Category	Option	Description
Toolbar	Compact	Toggles labels for each button in the interface, except the button itself.
	Details	Displays a pop-up to view and modify the track’s details.
	Save	Saves the player’s track against their profile with the current details and layout.
	Save as Copy	The same as Save, except a new copy of the track is saved.

	Undo	Reverts the last action.
	Redo	Restores the last reverted action.
	Test	Takes the player to a local race to test the track.
	Import	Displays a pop-up to provide track JSON data to load as a track into the builder.
	Export	Displays a pop-up with the current track's JSON to copy.
Visuals (eye)	Node	Toggles the visibility of all nodes.
	Wires	Toggles the visibility of the track wireframe.
	Path	Toggles the visibility of the track path itself.
	Materials	Toggles the visibility of all materials.
	Walls (Obstacles)	Toggles the visibility of all obstacles.
	Décor	Toggles the visibility of all decorations.
	Grid	Toggles the visibility of the movement grid.
Tools (hammer)	Select	Enables selection and movement of placed track elements.
	Pan Canvas	This is a placeholder for a panning feature, but it was never added.
	Path	Enables placing a new node on the canvas.
	Material	Enables the placement of a new material on the track. If a material is already selected, it adds a new point to the material.
	Wall (Obstacle)	Enables placing a new obstacle on the track. If an obstacle is already selected, it adds a new point to the obstacle.
	Decor	Enables placing a new decoration on the track. If a decoration is already selected, it adds a new point to the decoration.
Movement (four-way arrows)	Shift Nodes Forwards	Shifts all nodes one node backwards along the track.
	Shift Nodes Backwards	Shifts all nodes one node in the forward direction of the track.
	Flip Track	Reverses the direction of the track.
	Looped	Toggles the track between looped and open-ended.
	Recentre	Recentres all track elements to the centre of the canvas.
Node Options (node selected)	Width	Sets the track width in pixels around the selected node.
	Checkpoint	Toggles whether the selected node is a checkpoint to pass.
	Add Before	Adds a node sequentially before the selected node.
	Add After	Adds a node sequentially after the selected node.
	Delete	Removes the selected node from the track.
Material Options (material selected)	Type	Sets the material type of the selected material.
	Delete Material	Removes the selected material from the track.
	Delete Point	Removes the selected point from the selected material. If this is the last point, it removes the material entirely.

Obstacle Options (obstacle selected)	Delete Obstacle	Removes the selected obstacle from the track.
	Delete Point	Removes the selected point from the selected obstacle. If this is the last point, it removes the obstacle entirely.
Decor Options (decor selected)	Colour	Displays a colour picker to select the decoration colour.
	Delete Decoration	Removes the decoration from the track.
	Delete Point	Removes the selected point from the selected decoration. If this is the last point, it removes the decoration entirely.

***Table 5** – all track builder options and their functions, assigned by category.*

Table 6 lists the options available on the page and their functions, grouped by category. These will be discussed in more detail in the upcoming subsections. Options not shown in Figure 14 have also been included.

There are some options in the builder which use the same operations or extend others. For example, both the toolbar option “Undo” and the node option “Add Node After” involve placing nodes at an index, despite being different tools. To efficiently handle this redundancy, each operation is modularised as a distinct function at the top of the script. These methods are then invoked by whichever options or procedures require them. Additionally, the track is redrawn on the canvas each time it is changed.

6.2.2. Visualisation Options

```
let toggles = ["nodes", "wires", "path",
    "materials", "obstacles", "decorations", "grid"];

function applyToggleOnClick(target) {
    let button = document.getElementById(`visuals-${target}`);

    button.addEventListener("click", (e) => {
        let state = !renderer.getVisibility(target);
        toggleButton(button, state);
        renderer.setVisibility(target, state);
        renderer.redrawTrack();
    })
}

for (let target of toggles) {
    applyToggleOnClick(target);
}
```

***Code Sample 14** – the code to initialise all visualisation toggles in the track builder.*

Selecting each visualisation option toggles the visibility of its respective track component. This is achieved by setting the `TrackRenderer`'s corresponding `_visibility` key, which controls whether the track is drawn, as shown in **Code Sample 12**.

6.2.3. Tool Options

The builder tools are complex in that each one selects and handles a different combination of track elements. For instance, selecting the “Node” tool selects a single node, while the “Material” tool selects both materials and

the points within them. Additionally, the “Select” tool can select any track element, but must display the appropriate element options. Moreover, the “Material” tool has options to select the tool and to determine the material to place.

```
var tool = {Primary: null, Secondary: null};
var selection = {Primary: null, Secondary: null};
```

***Code Sample 15** – the tool and selection dictionaries used to handle tool and selection choices.*

The code handles this by using two distinct dictionaries for `Primary` and `Secondary` options, defined in **Code Sample 15**. These `Primary` and `Secondary` choices are then processed when the player interacts with the canvas, with the draw and canvas selection functions taking precedence where necessary.

6.2.4. Manipulation Options

Each manipulation option adjusts the nodes' order as described, then redraws the track on the canvas. For example, flipping the track reverses the order of the nodes.

During development, manipulating nodes proved frustrating and tedious. These options were not originally part of the requirements. However, they were devised to ensure corrective options are available when mistakes occur, such as skipping a node or putting the track the wrong way, without having to make several adjustments or redo the track. This is beneficial, as it means the player can reach their goal of building their track sooner than otherwise.

6.2.5. Element-Specific Options

Each track element type is distinct, each with a dedicated section for modifying its properties. The current element selection, described in **Section 6.2.3**, determines which interface is displayed. Changing these values updates the corresponding element, which is reflected on the canvas.

6.2.6. Toolbar Options

The “Details” option allows the player to edit the track's name and description. The track can then be saved using the “Save” option or saved as a copy using “Save As Copy”. A `POST /api/tracks` request is then made from the client to the server with the track data, and a response indicating whether the save was successful is returned to the client and displayed as a toast. Similarly, the player can use the “Test” option to try their track, which creates an offline lobby for the track with them as the only player. They can then select an option to revisit the builder once they are done, as detected by checking whether the last page the player visited was the builder.

Alternatively, track data can be imported and exported. This is achieved using the Track class's custom serialisation methods. Exporting the track displays a pop-up with the track converted to JSON. Imports are reversed: the player must provide valid track JSON data, which is validated during the `Track.toInstance()` method. The result is either loaded into the track, or the player is alerted that their format is inappropriate.

```
// functions/src/core/rendering/undo-redo-stack.js

undoAction() {
    if (this.getUndoSize() == 0) return;
```

```
    let action = this.#actions.UndoStack.pop();
    this.#actions.RedoStack.push(action);

    return action;
}

redoAction() {
    if (this.getRedoSize() == 0) return;

    let action = this.#actions.RedoStack.pop();
    this.#actions.UndoStack.push(action);

    return action;
}
```

Code Sample 16 *– the stacks used for the revision system.*

The “Undo” and “Redo” options (the “revision system”) are the more complex tools in the builder. When reverting changes, the system must handle various operations, not only additions but also modifications and deletions, as well as restoring previous revisions. Such a revision system can be modelled as stacks, where undone actions are kept in a redo stack for later restoration, with any subsequent undone actions added on top (Jakubec *et al.*, 2014). **Code Sample 16** shows these stack operations.

```
// public/js/page-builder.js

function recordAction(action) {
    reverter.recordAction(action);

    let undoBtn = document.getElementById("options-undo");
    let redoBtn = document.getElementById("options-redo");

    undoBtn.classList.toggle("disabled", reverter.getUndoSize() == 0);
    redoBtn.classList.toggle("disabled", reverter.getRedoSize() == 0);

    changesMade = true;
}

function placeNode(record, node) {
    track.addNode(node);
    renderer.redrawTrack();

    if (record) {
        recordAction({
            Type: ACTION_TYPE.Place,
            Target: node,
            Details: {
                Tool: {...tool},
                Selection: {...selection},
            }
        });
    }
}
```

Code Sample 17 *– the stacks used for the revision system.*

Code Sample 17 shows how the revision system records the action of placing a node. Each action includes the action type (defined as an Enum `ACTION_TYPE`), the element on which the action was performed, and details about the tool and selection when the action was carried out. The `tool` and `selection` dictionaries are copied to avoid interfering with the original variable reference when the revision system is handling an action. The action types of concern are placements, movements, and deletions. When undoing or redoing, actions are transferred to the respective stacks and executed. For example, undoing the deletion of a node restores it, while redoing the deletion removes it again, as if it were being deleted for the first time.

Any new actions which are performed clear all actions in the redo stack. While it is possible to build a revision system that tracks multiple “timelines” of actions, it is simpler to clear the stack with each new one (Jakubec, 2014). This is also a common approach among existing applications in different contexts, which players may be more accustomed to. Finally, if any actions are detected since the start of the session or the last save, if the player attempts to leave the builder, then they are prompted for confirmation to ensure they do not unintentionally lose their progress.

6.3. Track Listing

This section covers the Tracks List page.

6.3.1. Tracks List Overview

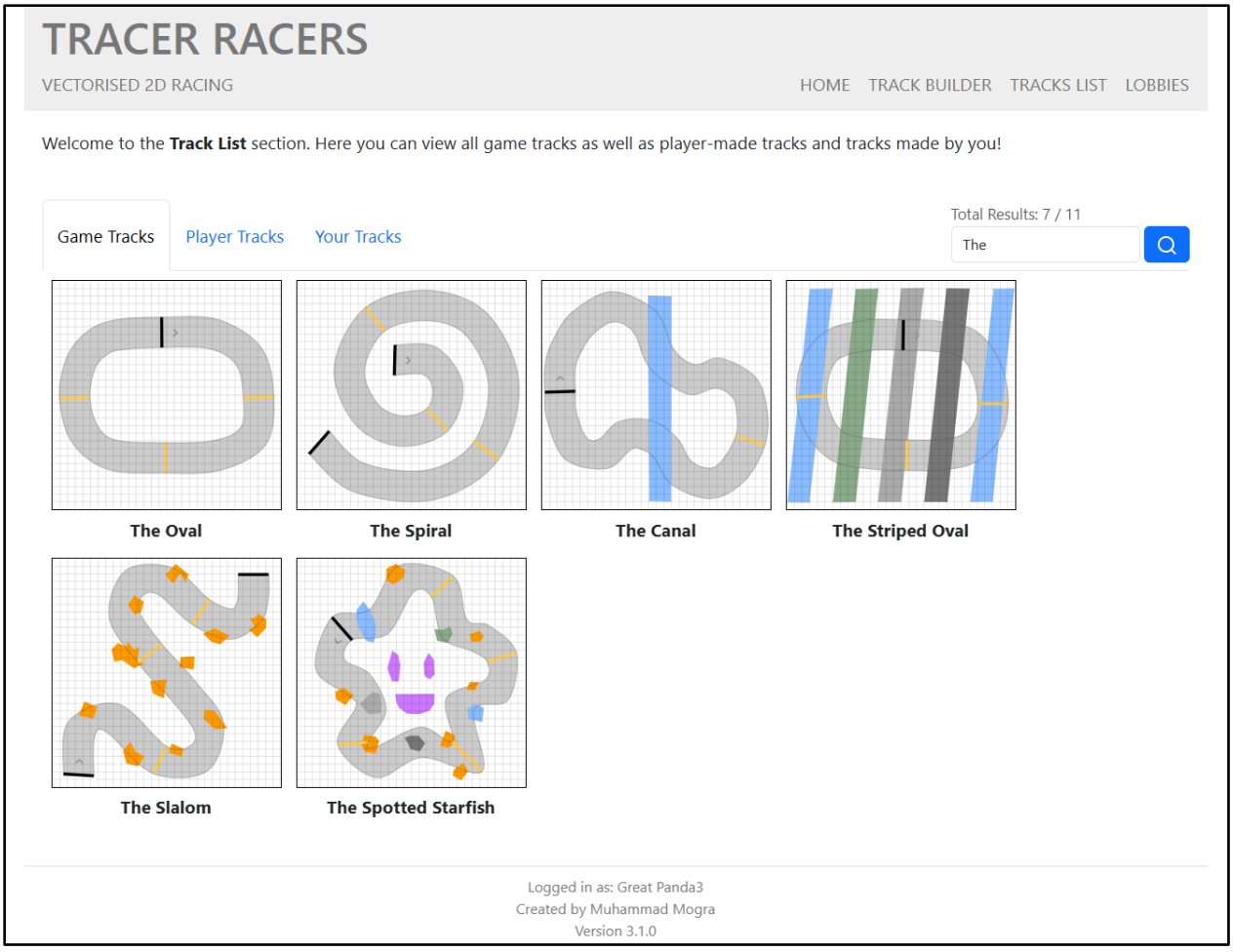


Figure 15 *– the tracks list with 7 game-made tracks with “The” in their title,*

1 of which has been made by another player.

In addition to being able to play tracks, it is also important for players to see them clearly to support the other objectives (**Primary Objectives 4 & 5**). *Vector Racer* and *PolyTrack* achieve this well by displaying their tracks as entries with previews, and by showing further details when selecting an entry. This provides players with sufficient information about a track option to decide whether they want to play it.

The Tracks List page lists all available tracks created by the game, players, or the player themselves. **Figure 15** shows the list of available tracks at the time of writing. Each entry displays a preview of the track rendered with a `TrackRenderer`. As with other games, tracks are organised into lists by who made them, with each list accessible via its tab. All available tracks are retrieved via the `GET /api/tracks` endpoint. The entries shown can be narrowed using the search bar, which limits them to those whose names match the query; the search count is the number of results found across all lists.

6.3.2. Track Details

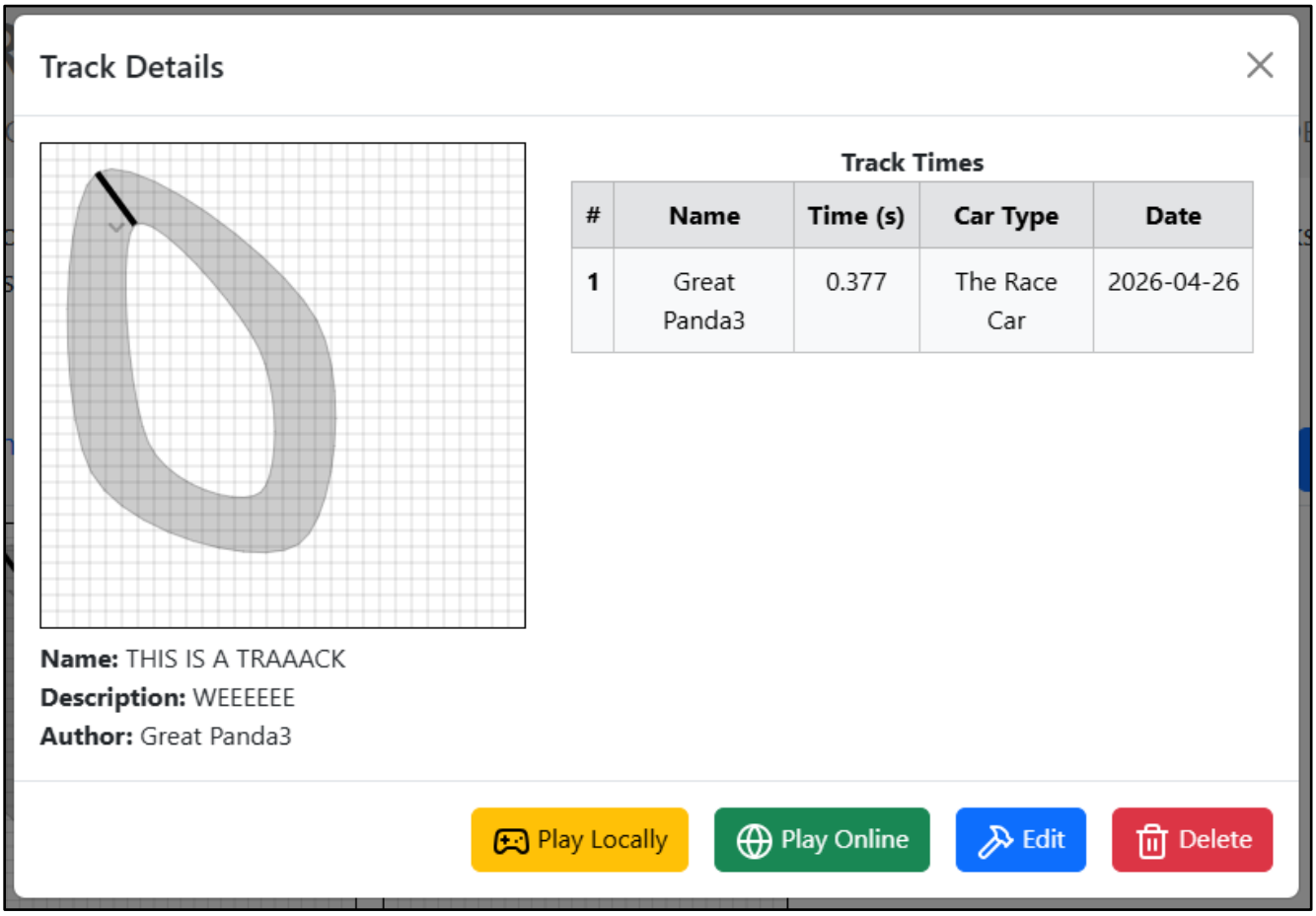


Figure 16 *– the entry of a track made by the current player with all lobby options. Additionally, the "Edit" and "Delete" options are available, as this is the player's track.*

Figure 16 shows the track details modal, which appears when selecting a track from the track list. Each modal displays a slightly larger preview of the track, along with the track name, description, and author. A leaderboard is also constructed, listing the players who have played the track and earned a high score. Displaying additional

information in a separate modal keeps the main track list less cluttered. These details are retrieved through `GET /api/leaderboards/:trackId`.

The modal also displays options to start an offline or online lobby for the track, discussed in more detail later. Each option invokes the `POST /api/lobbies` endpoint, adding the player as the only host. Additionally, the modal displays additional options based on the player's relationship to the track. Specifically, if the player is the creator of the track, they have permission to edit the track in the track builder or delete it, both of which are verified on the server by comparing their user ID to the track's official author ID. Selecting the edit option invokes the endpoint `POST /builder/:trackId` with the player's ID, and likewise for deletion, invoking `DELETE /api/tracks/:trackId`. Selecting the delete option prompts the player to confirm the deletion to reduce the chance of accidentally deleting a track.

6.4. Lobby System

6.4.1. Lobby Page Overview

The lobby page is the main area where players can manage their created session before starting the race. This gives them a chance to customise the race to their liking, tailoring their experience.

(section continues)

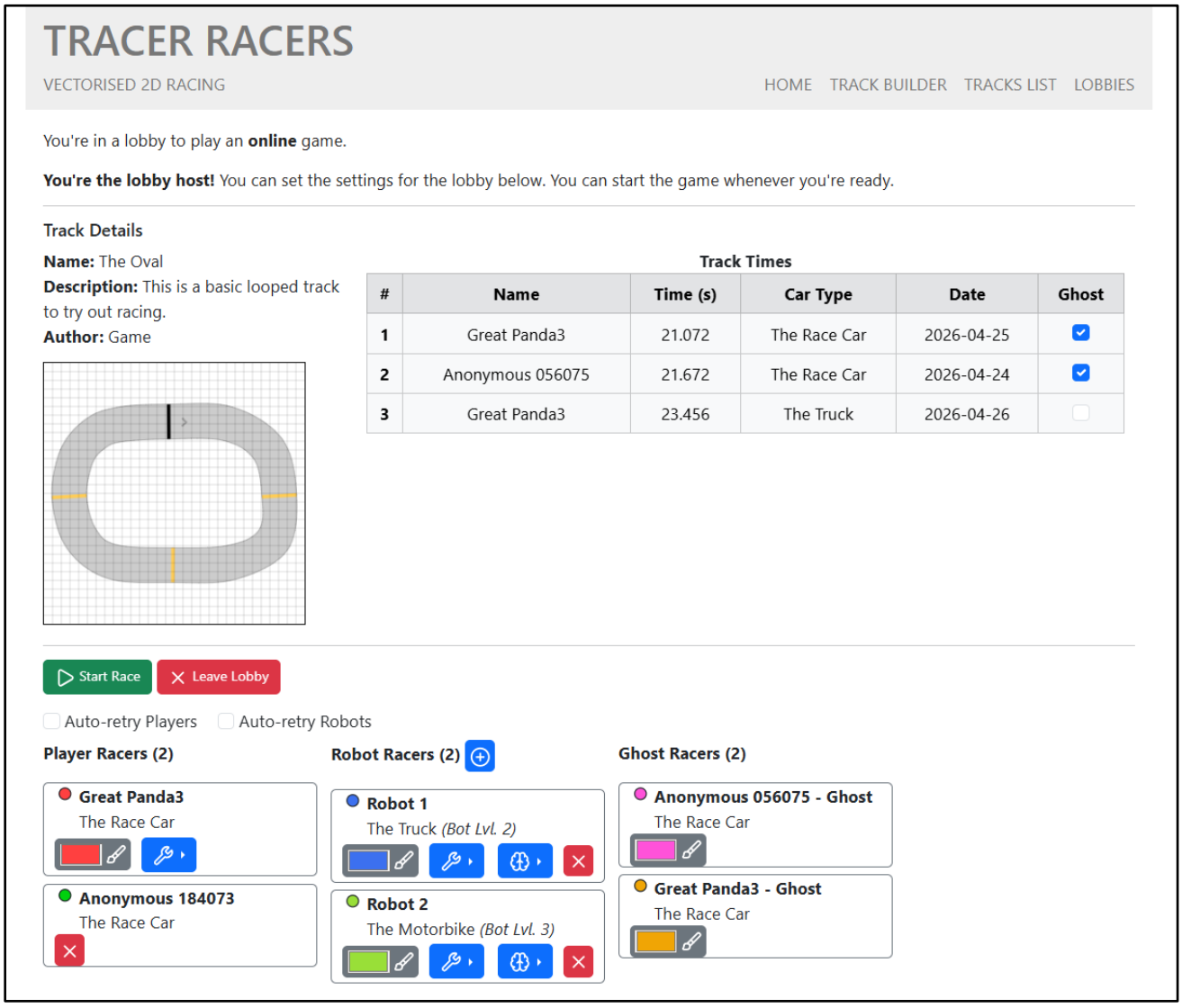


Figure 17 *– an online lobby from the perspective of the host. It has 1 anonymous player, 2 robots, and 2 ghosts.*

Figure 17 shows the lobby system. It displays the same information as a track entry in the tracks list, so the player can refer to it while editing the lobby. The leaderboard format is all the same, except for an additional column of checkboxes to select which ghosts to choose.

There is also a “Leave” button that allows players and hosts to exit the game. If a player leaves the game, their absence is propagated to the rest of the lobby occupants. However, the lobby will be disbanded entirely if the host leaves, returning all players to the Tracks List page. There is also an option to start the race, but this is restricted to the host. This invokes the `POST /api/lobbies/start/:lobbyId` endpoint, which verifies the requesting player’s ID against the host of the lobby to ensure only the host starts it.

Finally, the list of players currently in the lobby is displayed at the bottom of the page with additional lobby options. In an offline lobby, the host can add any number of local players, robots, and ghosts. They are also free to remove any of them from the race and modify any of their details according to their type. However, in an online lobby, the host can only add and modify robots and ghosts. Players who have joined the online lobby are free to change

their own details, but cannot adjust the rest of the lobby in any way. This restricts lobby permissions to the host while also allowing players to change their own details.

6.4.2. Player Details

Each player type includes different options and values, so they must be handled accordingly to ensure properties are available to change, such as robot level, or restricted, like ghost car types.

Player Type	Enumeration	Additional Values
Player	`PLAYER_TYPE.Player`	Colour, Car Type
Robot	`PLAYER_TYPE.Robot`	Colour, Car Type, Level
Ghost	`PLAYER_TYPE.Ghost`	Colour

Table 6 *– the available player types and the additional details they can take.*

Table 7 shows the implemented player types, their corresponding Enum values, and the details they include. Player information is stored as a player object that includes a player ID, name, and player type. The player type property allows other game logic to read what kind of player it is handling to process it accordingly, while the name is for human readability. While the player type is a defined Enum, IDs and names are derived differently for each player.

Human players will naturally have a player ID on their client, generated by the authentication system, so it is used directly. However, robots do not have an initial name or ID for reference, so their IDs are generated starting at 1 and incremented until the next available number is reached. As for ghosts, players can have multiple entries under their name for different car types, which makes it difficult to distinguish ghosts solely by ID. Considering this, ghosts are given a unique reference by concatenating the ID of the player who earned the score with the car type's name.

For names, players use the display name assigned to their account by the authentication system, while robots use the term “Robot” appended with their ID. Ghosts take the name of the original player associated with the score, with the term “Ghost” appended to distinguish it from a real player.

```
// functions/src/core/classes/gameplay/lobby.js

setPlayerDetail(playerId, detail, value) {
    Validator.keyPresence(this.#playerDetails, playerId);
    let entry = this.#playerDetails[playerId];
    entry[detail] = value;
}

// /public/js/page-lobby.js

lobby.setPlayerDetail(id, "carType", "The Race Car");
```

Code Sample 18 *– the dictionary of player details and how it is used.*

Once the player object has been constructed from the ID, name, and player type, it is added to the lobby. As mentioned previously, each kind of player has its own set of properties that can be adjusted. To manage this distinction, a separate `playerDetails` dictionary stores details by player ID, as shown in **Code Sample 18**. Using a dictionary approach to store values provides a scalable way to introduce new values in the future, as

the new key and value can be provided in the method. Any values not recognised or processed by the front-end or server are ignored.

6.4.3. Change Propagation

Players need to see the latest state of the lobby to identify their opponents and prepare for the race ahead. Hence, any changes made to the lobby should be reflected in other players' lobbies.

Whenever the host makes a change to the lobby, such as modifying a player detail or removing a player, the lobby is updated on the server via the `POST /api/lobbies/:lobbyId endpoint`. The server then saves these details to the Firebase RTDB.

```
let lobbyRef = ref(db, `lobbies/${lobby.getId()}`);

onValue(lobbyRef, (snapshot) => {
    let lobbyObj = snapshot.val() || null;

    if (!lobbyObj) {
        // redirect to race - if no race, will go back
        window.location.href = `/races/${lobby.getId()}`;
    }

    lobby = Lobby.toInstance(lobbyObj)

    refreshLobbyDetails();

    if (!lobby.getPlayers()[playerId]) {
        window.location.href = "/tracks";
    }
});
```

***Code Sample 19** – the Firebase `onValue` connection that updates the client's lobby data.*

After sending the changes to the server, these changes must be propagated to other human players in the lobby. This is achieved by creating an `onValue` connection assigned to the lobby reference in the RTDB, as shown in **Code Sample 19**. Whenever a change is made, a callback function is invoked with the updated snapshot. This snapshot is then used to update the client's version of the lobby data and the rest of their page to match what the host can see, subject to their permissions.

6.4.4. Lobby Lifecycle

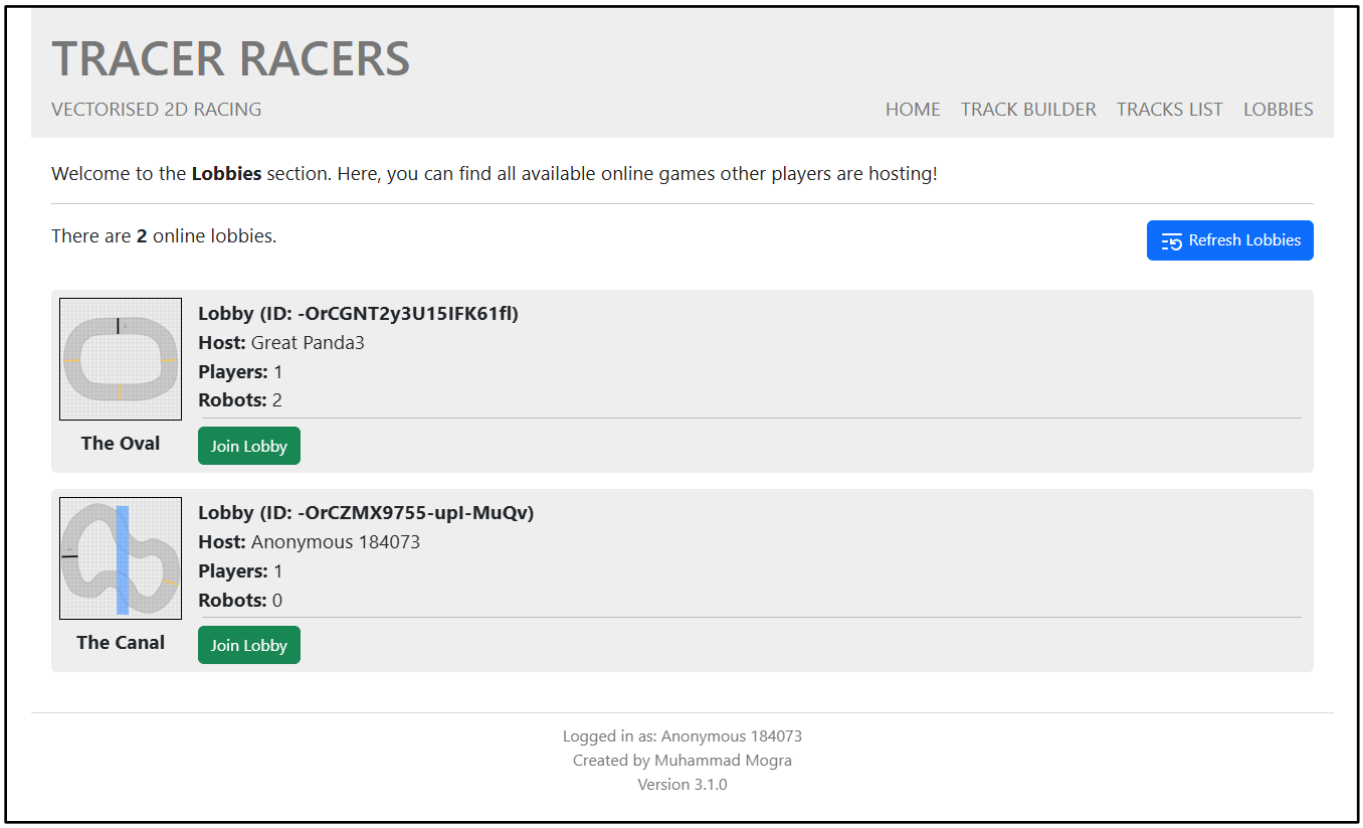


Figure 18 *– the Lobbies page with 2 online lobbies.*

(section continues)

Figure 18 shows the lobby list which players can use to view and join lobbies. Each entry is like a track entry on the Tracks List page, where a track preview is shown. The host, the number of players, and the number of robots are also displayed.

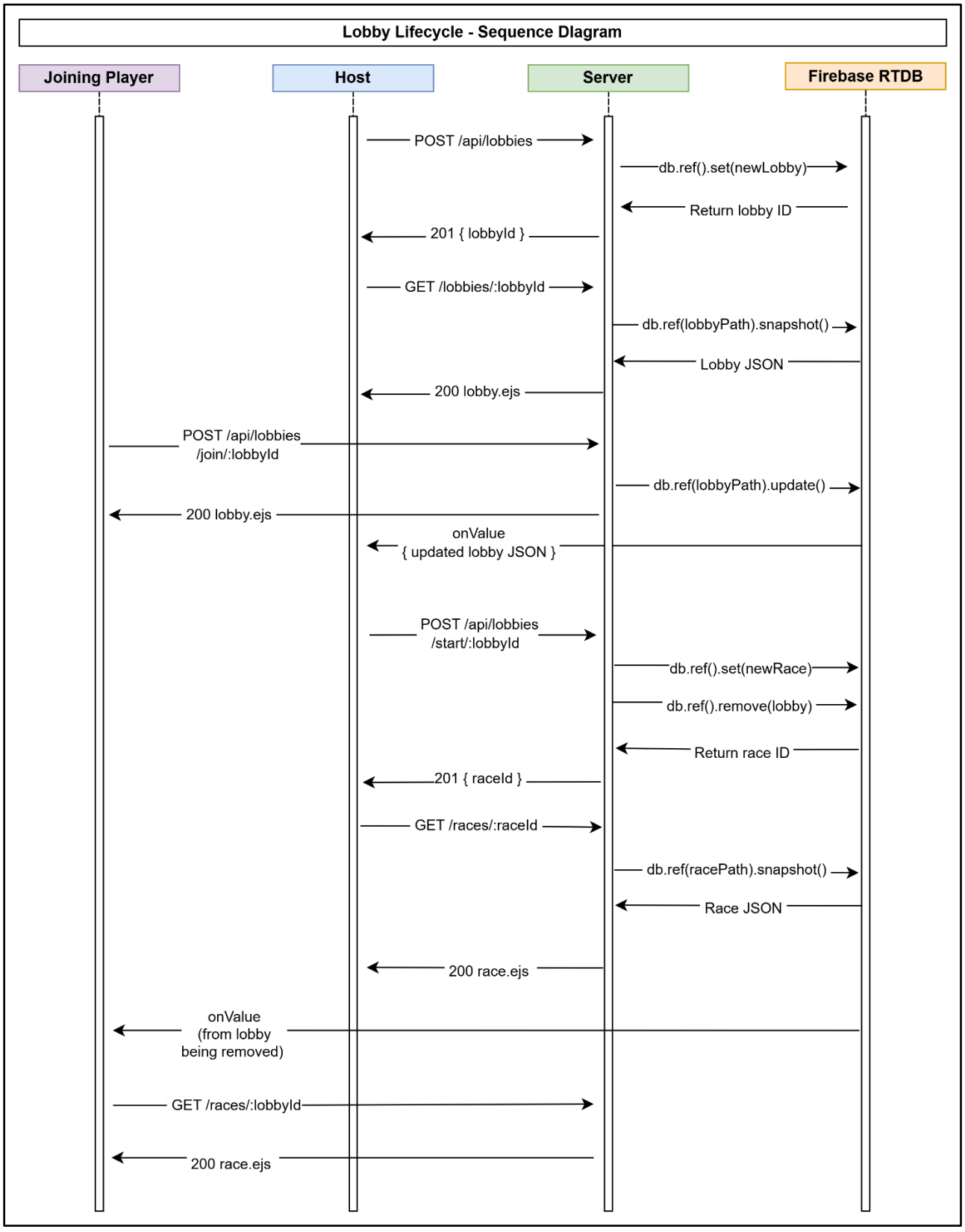


Figure 19 – a flowchart depicting lobby creation and joining for online multiplayer sessions.

Figure 19 is a sequence diagram representing the lifecycle of a lobby. The host first sends a request to create a lobby. The server processes this to create a new lobby that includes the host and sends back the lobby ID. The host then uses this ID to navigate to the Lobby page, where the details of the created lobby are displayed. Players wanting to join the lobby submit a request via `POST /api/lobbies/join/:lobbyId`, which the server then processes to update the RTDB lobby reference with the player's presence and return the lobby ID to the player so they can join. The `onValue` connection on the host's side receives this update and displays the player's presence. The host can then start the lobby by invoking `POST /api/lobbies/start/:lobbyId`, which creates a new race using the same lobby ID and removes the lobby. Removing the lobby redirects both the host and the player to the Race page of the newly created race instance.

6.5. Race Gameplay

This section covers elements related to core gameplay and its design.

(section continues)

6.5.1. Race Page Overview

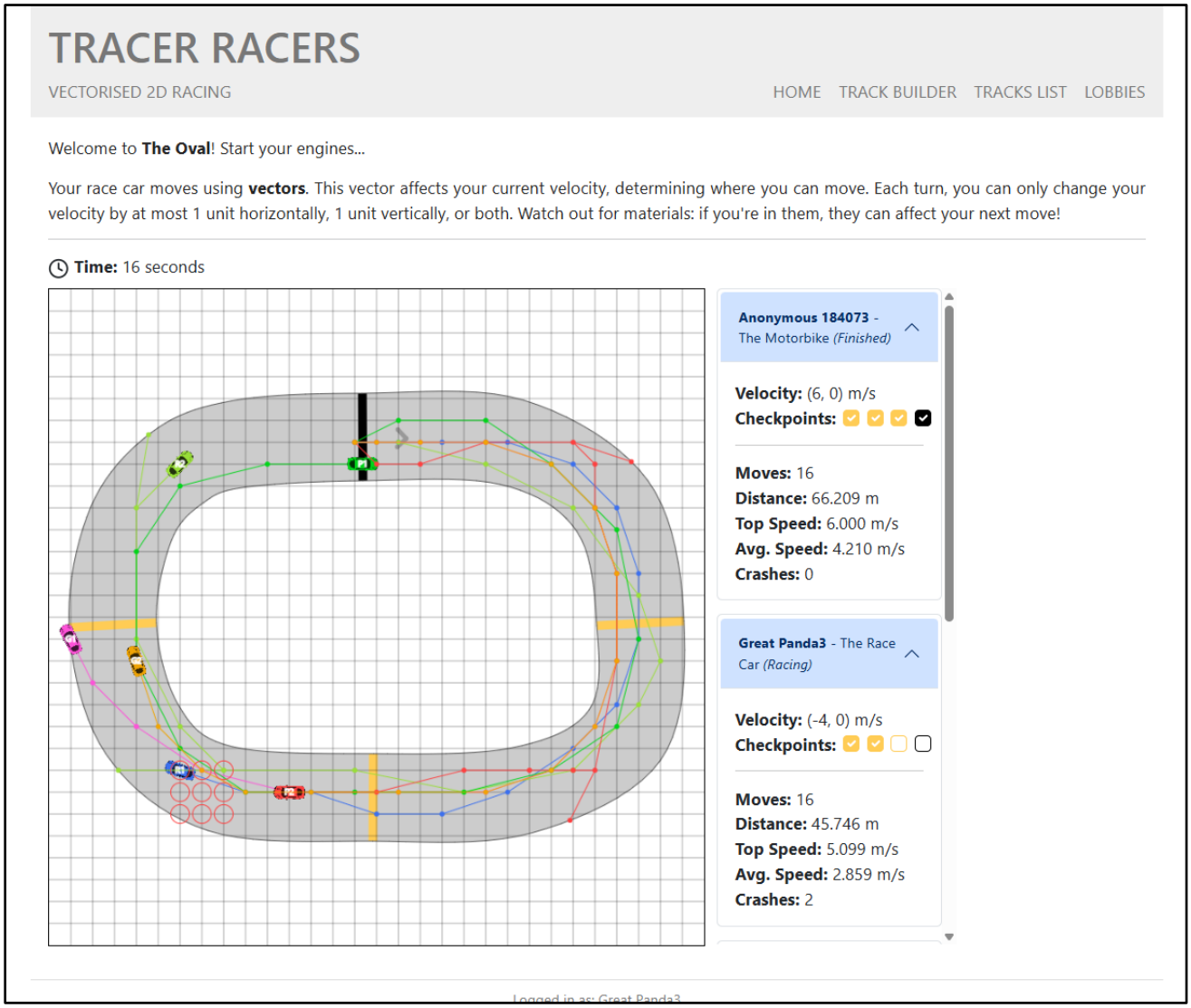


***Figure 20** – an online race in action with 2 players, 2 robots, and 2 ghosts.*

Figure 20 shows a race midway through. The page presents the track the host had intended to play with all players starting from a single point at the starting line. The track is rendered by a `TrackRenderer` on the client, like previous use cases. Only components related to racing and gameplay are rendered, namely the track path, checkpoints, track outline, and grid. An additional renderer, `RaceRenderer`, is used to render race cars and their paths, as well as the current player's available moves.

Each race is managed by a `RaceInstance` object, which consists of all information about the race and player states, as well as methods to gather valid moves and determine collisions.

6.5.2. Player Move Selection

As in *Vector Racer*, players race around tracks by selecting vectors visualised over a selection grid. This was used for the application as well, as it seems like an intuitive way to demonstrate where the player is allowed to

move. The movements they can make are determined by their race car type and the material they sit on. If players crash, they restart from their last location with zero velocity or end the race. Other players will still be able to continue until the race is over.

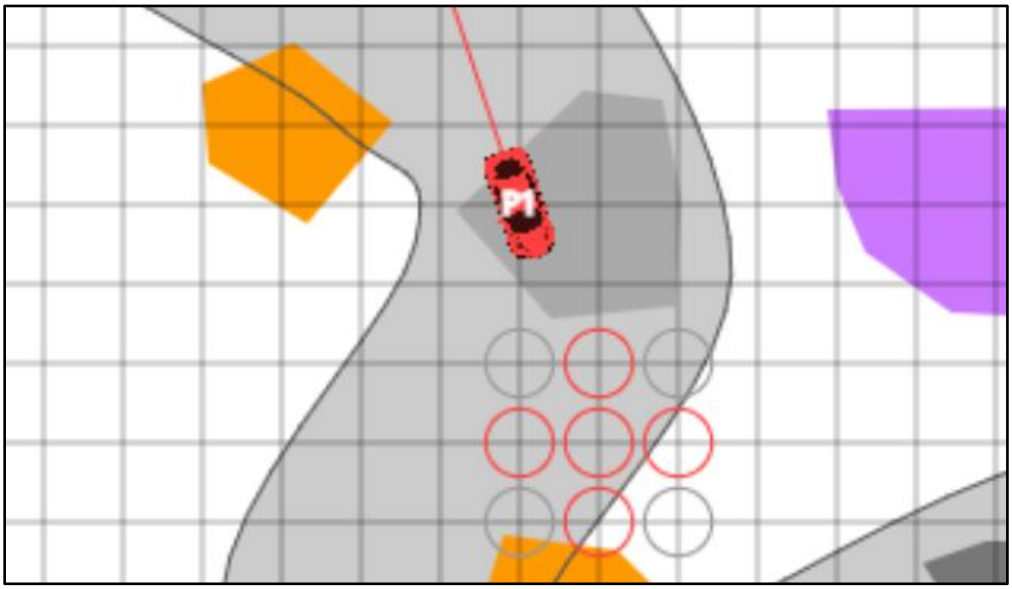


Figure 21 *– a race car on "Curb". Its moves have been limited to a single option.*

```
// functions/src/core/classes/gameplay/race-instance.js

for (let material of materials) {
    let crosses = 0;
    let edges = material.getEdges();

    for (let edge of edges) {
        let line = new Line(edge[0], edge[1]);
        let crossed = ray.getIntersection(line);
        if (crossed) crosses += 1;
    }

    if (crosses % 2 == 1) {
        onMaterial = material;
        break;
    }
}
```

Code Sample 20 *– the ray-casting algorithm being applied to materials.*

As mentioned when discussing track representation, players on materials or in obstacles must be detected. The ray casting algorithm (Hormann & Agathos, 2001) is a procedure for determining whether a point lies inside a polygon, based on how many intersections it makes with its edges from the inside: an odd number of intersections indicates it is inside. The method fits this context because race cars are represented as single points, and materials and obstacles are represented as polygons. **Code Sample 20** demonstrates how this is performed for materials, where obstacles follow a similar process.

Materials determine the moves players can make, while obstacles cause players to crash. The logic for obstacles is simpler than materials, as all that needs to be done is to mark the player as crashed when they intersect any

of the obstacle's edges. This is more complex for materials, however, as there also needs to be information on how to restrict the player's moves. Moreover, this restriction should apply to all car types.

```
// functions/src/core/enums/material-info.js

const MATERIAL_INFO = {

    ["Curb"]: {
        Id: 1,
        Name: "Curb",
        Description: "Limits the moves the player can make",
        Colour: "#AAA",

        Effect: (racecar, x, y) => {
            let acc = Math.abs(racecar.getMaxAcceleration().x);
            return {
                Offset: new Vector2(x, y),
                Active: Math.abs(x) + Math.abs(y) <= acc
            }
        }
    },
    ...
}
```

Code Sample 21 *– the definition of the material "Curb", including the effect.*

```
// functions/src/core/classes/gameplay/race-instance.js

let moves = [];
let acc = racecar.getMaxAcceleration().x;
let effect;

if (onMaterial) {
    let type = onMaterial.getType();
    effect = MATERIAL_INFO[type].Effect;

} else {
    effect = (racecar, x, y) => { return { Offset: new Vector2(x, y), Active: true } }
}

for (let x = -acc; x <= acc; x++) {
    for (let y = -acc; y <= acc; y++) {
        let move = effect(racecar, x, y);
        moves.push(move);
    }
}
```

Code Sample 22 *– the procedure which applies the effect of the material.*

This is managed using a dictionary of all available materials, along with a function that defines their effect. **Code Samples 21 & 22** show an example of how the material "Curb" is defined and how it is applied to restrict the player's moves to those shown in **Figure 21**. Using this structure allows materials to be scaled simply by creating a new definition and restricts the moves of any car type by its max acceleration, as intended. Each move within the player's acceleration range is checked and added to their list of available moves with details of the

acceleration and whether the move can be made. Seeing both active and inactive moves helps the player understand their car's maximum potential and how much they have been affected.

6.5.3. Robot Pathfinding and Difficulty

Robot racers are autonomous, driving around the track on their own accord. In line with the objectives, there also needs to be a way to adjust the robot's intelligence based on the level specified in the lobby. The implemented algorithm for robots is a score-based approach in which each cross-section of the movement grid has a calculated score. The score is a function of how far along the track the point is and how close it is to the centre line. This leads to behaviour in which the robot tries to make as much progress as possible, using the centre line as a kind of racing line.

```
// functions/src/core/classes/gameplay/robot-determiner.js

function shorterDirection(current, next) {
    let count = centre.length;
    let toRight = (next + count - current) % count;
    return toRight > count / 2 ? -(count - toRight) : toRight; // left / right
}

function getMoveScore(move) {
    let pos = racecar.getPosition();
    let oldUiPos = pos.scale(spacing);
    let initial = getClosestSegmentIndex(oldUiPos);

    let predict = raceInstance.predictMove(racecar, move.Offset);

    let newPos = predict.Position;
    let newUiPos = newPos.scale(spacing);
    let closest = getClosestSegmentIndex(newUiPos);

    let segment = centre[closest];
    let distance = segment.toPoint(newUiPos);
    distance = Math.max(distance, 1);

    let progress = shorterDirection(initial, closest);
    let score = Math.log(1 / distance) + progress;

    let favour = favours[predict.State];
    if (favour) score += favour;

    return score;
}
```

***Code Sample 23** – the method used to calculate the score of a given move.*

Code Sample 23 shows the scoring function. Recall from the discussion about track representation that boundaries, including the centre line, are constructed from tiny segments. Thus, for any given move, progress is determined by the index of the closest segment to the new move relative to the old one. Calculating this progression is simple for open-ended tracks, as it is only a matter of predicting the next move and finding the difference between the indices of the two closest segments. However, this is more complex for looped tracks, as at the start of the race, the robot will think its initial moves behind the racing line provide the most progress by going alongside segments with higher indexes, but this is not the case and will cause it to drive backwards. To

resolve this, segment indexes are made unidirectional using `shorterDirection()` in **Code Sample 23**, so that a segment, its start, and its end cover the entire track rather than skipping the starting line. Finally, the score is adjusted by a fixed amount, if applicable, rewarding it for crossing the finish line and penalising it for crashing by adding or removing a fixed value.

```
// functions/src/core/classes/gameplay/robot-determiner.js

let deterministic = level >= 4;
let maxDepth, depth, randomRisk;

if (deterministic) {
    maxDepth = 3;
    depth = 3;
    randomRisk = 0;
} else {
    maxDepth = level;
    depth = Math.ceil(Math.random() * maxDepth);
    randomRisk = 1 / 2**(level == 0 ? 1 : level);
}

function runRecursion(d) {
    let moves = raceInstance.getPossibleMoves(racecar);
    let bestScore = -100;

    for (let move of moves) {
        if (!move.Active) continue;

        let score = getMoveScore(move);

        if (d > 0) {
            let originalPos = racecar.getPosition();
            let originalVel = racecar.getVelocity();

            let next = raceInstance.predictMove(racecar, move.Offset);

            if (next.State == RACECAR_STATE.Crashed) { // simulate crash
                racecar.setPosition(originalPos);
                racecar.setVelocity(new Vector2());

            } else {
                racecar.setPosition(next.Position);
                racecar.setVelocity(next.Velocity);
            }

            score += runRecursion(d - 1);

            if (Math.random() < randomRisk) {
                score += Math.abs(score) * 0.25;
            }

            racecar.setPosition(originalPos); // hacky reset per move (TODO)
            racecar.setVelocity(originalVel);
        }

        if (score > bestScore) {
            bestScore = score;
            if (d == depth) finalMove = move.Offset;
```

```
        }
    }

    return bestScore;
}

runRecursion(depth);
```

***Code Sample 24** – the recursive procedure to find which series of moves has the best score.*

The score calculation method is part of a larger recursive method that the robot uses to find the next best move. **Code Sample 24** shows the corresponding code, which performs a breadth-first traversal by calculating the score of the first set of moves, then those that branch from it, and so on. The robot's level determines the depth of its vision. Robots at a lower level are also more susceptible to having their scores randomly inflated, which could lead to better or worse moves. This influence is multiplicative, so it scales accordingly to maintain its effect on larger scores.

The methods exist in a class called `RobotDeterminer`. Robot moves are executed on the server using the `RaceInstance` to analyse the track. Once the move is calculated, it is returned to the server for their move to be made as a player would.

6.5.4. Ghost Move Selection

Like robots, ghosts also move on their own accord. However, a ghost's moves are predetermined by those stored in the corresponding score entry it exists from. When adding a ghost to a lobby, the moves are extracted from the entry and added as part of the ghost's details. These moves are then read during race processing and used, with each move corresponding to a unit of elapsed time. Once a move has been selected, the race instance object applies it in the same way as player and robot moves, ensuring consistency with game logic. As the move follows game logic, the ghost will be marked as "Finished" when its last move crosses the finish line, so that it will be skipped during turns.

6.5.5. Turn System

Each player in the race takes their turn one at a time, and the turn is processed according to their player type. As mentioned before, robot and ghost moves are determined on the server, so they automatically move on to the next turn. For players, the set of available moves is displayed on their screen depending on whether it is their turn. Their move is then submitted via `POST /api/races/:raceId/move`, which updates the race reference in the Firebase RTDB. This triggers the corresponding `onValue` connection to the reference for each client, which displays their moves if it is their turn for the cycle to repeat. All moves are added as pending moves to the race instance, which are executed all at once at the end of a full turn cycle.

```
function processRaceMove(raceInstance, racecar, move) {
    // "move" is technically acceleration

    let playerType = racecar.getPlayerType();

    while (raceInstance.isAnyRacecarInState(RACECAR_STATE.Racing)) {
        if (playerType == PLAYER_TYPE.Player) {
            // add player move
```

```
        } else if (playerType == PLAYER_TYPE.Robot) {
            // add robot move

        } else if (playerType == PLAYER_TYPE.Ghost) {
            // add ghost move
        }

        if (move) {
            raceInstance.addPendingMove(racecar, move);
        }

        raceInstance.findNextTurn();

        let nextRacecar = raceInstance.getCurrentPlayer()

        if (nextRacecar.getPlayerType() == PLAYER_TYPE.Player) {
            break;

        } else {
            racecar = nextRacecar;
            playerType = racecar.getPlayerType();
        }
    }
}
```

Code Sample 25 *– the gameplay loop to prompt the next player for their move.*

Code Sample 25 shows a condensed version of the turn system. The loop continues until there are no active racers, at which point the client script for the Race page displays the game's results. Players who have retired or finished are skipped, as they are no longer partaking in the race. As the turn system is event-based and primarily triggered by the first player, the server will process a few turns first until it finds a suitable human player when creating a race to prevent the race from locking the player out of making their move.

6.5.6. Race Results and Leaderboards

Players can see a set of live race statistics for each player in the race, as shown earlier in Figure X. These details include metrics on their speed, checkpoints, and status, stored in `RaceStats` objects and surfaced on the page. Each time the turn changes, the `onValue` connection associated with the race reference in the RTDB updates all statistics.

(section continues)

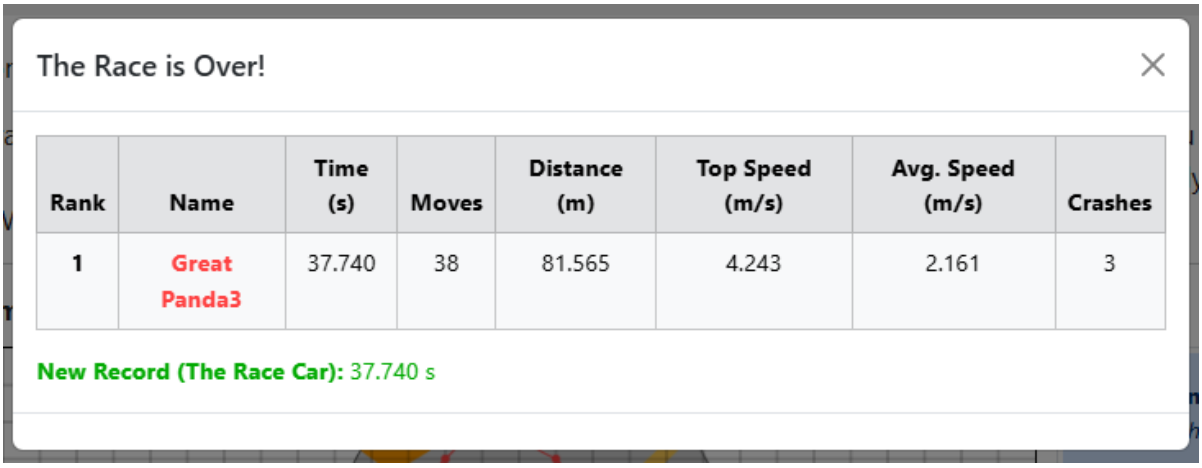


Figure 22 *– the race results modal at the end of a race with only 1 player.*

Once all players have either finished the race or retired, each player's race statistics are compiled into a single leaderboard, shown in **Figure 22**. If the current player has set a new high score, a notification appears at the bottom of the modal showing their time. Similarly, if they have beaten their previous score, then the new score is displayed alongside the old score and the difference between them. Scores are ordered by time and coloured according to the player's state: white if the player has finished, yellow if they are currently crashed, or red if they have retired. Retired player times are placed at the end of the leaderboard, but are instead sorted by how long they lasted in the race. Players who have not signed in with an account can see a button which prompts them to sign in to save their score.

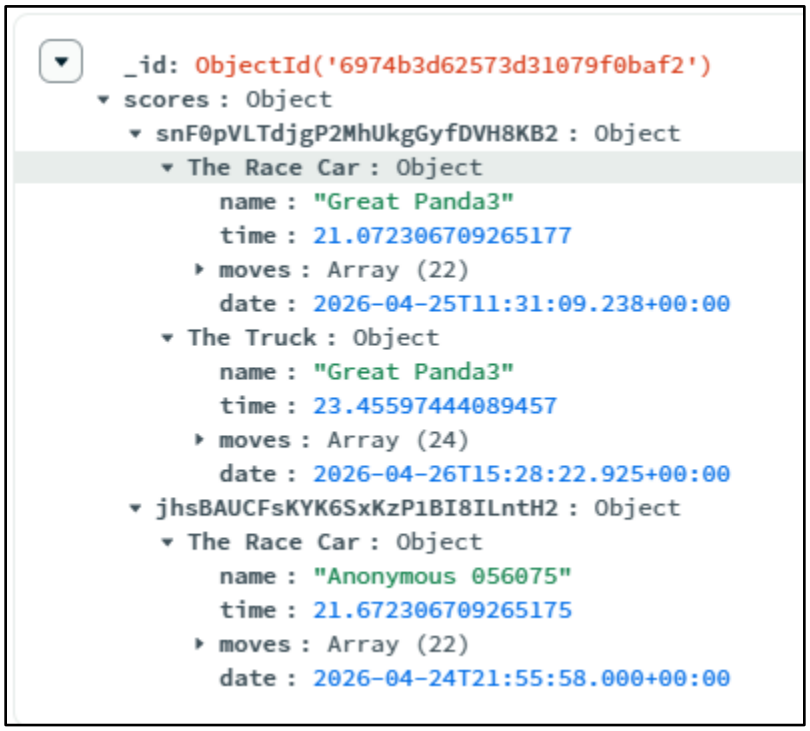


Figure 23 *– the leaderboard scores for the track "The Oval".*
The player "Great Panda3" has 2 entries for different car types.

Leaderboard scores are stored as entries assigned to a player ID. A player can have several times for different car types for the same track, as shown in **Figure 23**. This is because some race cars are more able than others, leading to drastically different times. Equally important is knowing whether a player has achieved a very good time in a slow car, demonstrating some level of skill.

7. Testing

Vitest is an NPM package used to write tests for the application in `/tests`. These were primarily written at the end of each Epic as defined in the requirements table. This was done to ensure both optimal workflow and consistent core functionality between versions. Most tests were written per class in `functions/src/core`, with a set of tests per method to assess varying outputs and edge conditions. All tests serve to ensure game logic is upheld, more so than to ensure efficiency.

```
// tests/classes/geometry/vector2.test.js

describe("classes.vector2.mag", () => {
    it("calculates the magnitude of a positive Vector2 (X, Y)", () => {
        expect( new Vector2(5, 3).mag() ).toBeCloseTo( 5.83095189485 );
    });

    it("calculates the magnitude of a negative Vector2 (-X, -Y)", () => {
        expect( new Vector2(-5, -3).mag() ).toBeCloseTo( 5.83095189485 );
    });

    it("calculates the magnitude of a zero Vector2 (0, 0)", () => {
        expect( new Vector2().mag() ).toBe( 0 );
    });
});
```

***Code Sample 26** – a set of tests to ensure `Vector2.mag()` calculates magnitude properly.*

Code Sample 26 shows a collection of tests for one method of the Vector2 class. Each test follows a convention named after the class and method being tested, followed by specific scenarios outlining what is being tested. The format offered by Vitest allows tests to be structured and human-readable, making the development experience more enjoyable. As of the application submission, all 210 tests across 18 test files pass.

8. Critical Appraisal & Future Work

This section analyses the work done and compares personal skills to those at the start of the project. It also identifies areas for improvement in the project as future work.

8.1. Evaluation of Primary Objectives

8.2. Primary Objective 4.1

Players can view a list of tracks to choose from, as well as create their own lobbies with customised settings. They can then start the lobby to begin the race, which allows them to take their turns one at a time. The rules of the track conform to those of the original Racetrack game and extend it by including several car types. The track, race cars, and race car paths are visualised to help the player see the game state and plan their next move.

Materials effects and collisions are applied to cars to add a challenge by affecting their movement or causing them to crash. Live statistics are also displayed alongside gameplay to inform the player about the game state, such as performance and the number of moves made. All requirements associated with **Primary Objective 4.1** have been fulfilled and tested.

8.2.1. Primary Objective 4.2

Players can access a track builder with tools to create their own track with points, materials, obstacles, and decorations. Tracks are represented as Catmull-Rom splines, which form organic curves that can be toggled between looped and open-ended to produce a wide range of track forms. Visualisations can help the player see their track's widgets, adjust them, or remove them to see what the track really looks like. Details associated with the track, such as the name and description, can be modified. A suite of controls for easily manipulating the track is available if the player makes a mistake or wishes to restore a previous state. All requirements associated with **Primary Objective 4.2** have been fulfilled and tested.

8.2.2. Primary Objective 4.3

From the Track Builder page, players can save their tracks as new ones or as copies. Players can also test their tracks in the builder to assess how they feel to play and adjust them accordingly. Players can also view their tracks in the Tracks List, from which they can choose to continue editing them or delete them. Track details can also be changed and resaved. All requirements associated with **Primary Objective 4.3** have been fulfilled and tested.

8.2.3. Primary Objective 4.4

Players can create a local lobby to race against others on the same machine. Starting a lobby puts all players in a race, taking turns on the same machine. The application can support multiple players racing at once and allows the host to adjust each player's details before a race begins. Players can also race against previous high scores for a track in the form of ghosts, which autonomously play out the scorer's moves as the race goes on, presenting a benchmark both in time and visually. All requirements associated with **Primary Objective 4.4** have been fulfilled and tested.

8.2.4. Primary Objective 4.5

Players can choose to add robot racers to their lobby before starting a race. Robot racers can calculate their moves, albeit slowly, with great difficulty, and perform them to race around the track, providing a reasonable challenge. Robot racers are treated the same as regular players, with their race stats being displayed as well as their associated visuals. All requirements associated with **Primary Objective 4.5** have been fulfilled and tested.

8.3. Evaluation of Secondary Objectives

8.3.1. Secondary Objective 1

In addition to creating local lobbies, players can create online lobbies to race against players on different machines or networks. Online lobbies have the same customisation capabilities as offline lobbies, except that only joining players can customise their car. Players can race together, with all features managed accordingly, as in local racing. Players who earn high scores also have their performance saved against the track and their

race car, adding a layer of competitiveness that other players can use as ghosts. All requirements associated with **Secondary Objective 1** have been fulfilled and tested.

8.3.2. Secondary Objective 2

There are 4 robot difficulty ratings for players to choose from, which players can set in a lobby before a race. The difficulty of the robot racers directly affects their performance by influencing their scores and move-prediction depth, with higher-difficulty robots performing noticeably better than lower-difficulty ones. All requirements associated with **Secondary Objective 2** have been fulfilled and tested.

8.3.3. Secondary Objective 3

Players can personalise their race car in the lobby before starting a race by selecting a car type and a colour. Colours can be selected for all player types and, where applicable, for car types. All requirements associated with **Secondary Objective 3** have been fulfilled and tested.

8.4. Project Impact

The application requires players to race around tracks using vector principles, where they can only increase their speed by 1 unit in either the X or Y direction. Given its mathematical nature, it could be used as an educational tool to teach how vectors work interestingly and could help learners to plan with these restrictions in mind.

As for what it takes to run the application, the Firebase Blaze plan and MongoDB Atlas are both free services. However, this is only for very small applications. Larger player sizes would incur higher costs and require better plans due to the network bandwidth, memory, and computing power the application would use. Hence, as the application evolves, it could incur greater costs, so it would be useful to devise strategies to improve its performance and efficiency, reducing the resources required to operate.

8.5. Personal Development

This section concerns my personal development throughout the project. At the start, I filled in a skills log with ratings for certain skills.

Computing and research skills		
Skill	**How confident do you feel with this skill? Rate 0-5 (never encountered to mastery).**	
	Before	**After**
Researching information	4	3
Reading and understanding scientific papers	3	3
Referencing and citing sources accurately	4	4
Presenting data in figures	3	3

Analysing and interpreting data	4	4
Scientific communication (written and oral presentations)	3	3
Programming in a high-level language (e.g. Java, Python, JS, C/C++, Rust, etc.)	4	4
Software design (how to choose design pattern to implement functionalities of a software tool)	3	3
Testing and Evaluation (how to test a software tool and choose appropriate metrics for its evaluation)	3	4
Algorithms (how to design/choose appropriate algorithms and data structures to efficiently solve a computing problem)	4	4
Career readiness (you have the fundamental technical knowledge (coding skills, managing data, design, etc.) and soft skills (communication, problem solving and resilience, etc.) to apply for jobs, successfully land roles, and perform effectively in industry)	3	3
Problem solving (overcoming an unexpected challenge or issue)	4	5

***Table 7** – my skills log before and at the end of the project.*

Table 8 shows my responses at the beginning of the project compared to now. Most figures remain the same or have gone down, as I do not feel I practised them as much as I would have liked for the project. For example, most of the preliminary reading I did was from websites and state-of-the-art sources. At the same time, I found only a few published papers that could be used to devise some of the algorithms for the application. There are also some, I think, that I have not quite done as well as I could, particularly in the application's design with respect to the requirements in later versions. This is discussed in more detail in **Section 8.6**.

However, there are also some improvements, particularly in testing and problem-solving. While testing was not perfect, as will be discussed, it was my first time using a testing framework like Vitest, and I found it very useful for ensuring game logic remained consistent throughout development. I also feel my problem-solving skills have improved, as there were many concepts which were new to me that I had to understand, primarily splines, Express in Node.js, and Firebase tools. Despite the relatively consistent scores, I do feel like I have learned a lot from undertaking the project, especially how far I can push myself, even when some concepts seemed very difficult. It is nice to look back on things I had almost forgotten I worked on to see how far I have come.

8.6. Improvements & Future Work

As mentioned before, the application has fulfilled all primary and secondary objectives stipulated for the project, as well as all specified requirements. However, there are many improvements to make to the project.

An overarching factor of the project was time management. Progress in the earlier stages of the project was promising, as Version 0 and Version 1 were both completed on schedule. However, this momentum eventually

slowed for Version 2 and Version 3 due to complacency, causing significant delays in their development. While the project's goals were met according to the requirements, this led to a lack of research in certain areas, leaving the solutions less well-informed.

In terms of the application, one of the most important factors is endpoint security. In the application, many data-related endpoints, such as track creation or deletion, accept the player ID as a query parameter or in the request body. While each player ID is unique and can be difficult to remember, public access to the API means any user could maliciously submit a request using tools like Postman. Notably, the only validation performed for such operations is a comparison of the submitted player ID with the resource's ID. This means bad actors could delete other players' content by knowing the target track ID and player ID with no restrictions. The system could be improved with an additional layer of authentication, such as tokens, which warrants further investigation (Kader & Ahmed, 2025).

The application could also be improved by using more advanced technologies. For instance, MongoDB Atlas is used to host a separate MongoDB database, while Firebase includes the same NoSQL database. Although MongoDB was originally preferred due to its high data limit (MongoDB, 2026), it may be better to use Firebase for a unified system, given the choice to use Node.js for the back end, since the front end uses JavaScript. Additionally, the front-end could use a different technology to update content dynamically. At the start of the project, EJS was useful to pass data to web pages to load content with embedded tags. However, as familiarity with the project grew and more advanced features were used, the benefit of embedding data on the page was outweighed by `fetch` calls. This transition eventually made EJS useful only for consistency in its partials, but not much else. In hindsight, a framework like React would seem beneficial for displaying dynamic content such as tracks and lobbies, and could improve the application (Raul *et al.*, 2025). For type checking, TypeScript could have been used as an alternative, with even more rigorous type restrictions, to make class definitions much clearer. This is slowly becoming more popular than JavaScript and would have helped development (Sharma & Sharma, 2026).

Another factor concerns gameplay, particularly in robot racers. It may have been noted that the breadth-first traversal runs in exponential time. As a result, playing with several low-level robots is feasible, but even a single high-level robot causes a noticeable freeze while the server processes moves. The poor quality of the algorithm is due to implementing a somewhat working version during development without planning for optimisation, such as using Dijkstra's algorithm with Fibonacci heaps to prioritise the best scores first before processing others for $O(E + Vlog(V))$ time complexity, a significant improvement (Chen, 2003; Fredman & Tarjan, 1987). Elements which would support heuristics, such as calculating the official racing line around the track rather than using the centre line, were also considered, but again omitted due to time constraints. This would be a key area to improve for gameplay, ensuring the server does not hang while processing multiple high-level robot moves simultaneously and enabling players to play against robots.

Other gameplay factors could also be improved, such as the level of customisation available for players. Currently, all racers share the same sprite, but unique images could be used to represent different car types more accurately, such as a motorbike for "The Motorbike". Additionally, the number of available race car types and materials is limited, and more could be added. Racecar types are especially unbalanced, as "The Motorbike" has statistics that far outweigh those of the other two types. The game could be improved by adding features that make it more enjoyable and highlight its existing strengths.

Although the requirements are met, a few features do not yet meet them and still need to be implemented. For instance, race data is not deleted when a race is finished, leading to an inevitable buildup if not removed and incurring the costs mentioned earlier. Many elements of the application are also untested. Requirements in earlier versions, particularly Version 0 and Version 1, focused primarily on game logic that ran locally, making it easy to simulate scenarios and run tests to cover edge cases. However, as the application became more complex, this

testing became scarcer and harder to manage, especially for older tests that needed updating. Some tests do exist for the latter requirements, but these are not extensive. More tests could be written to cover the new features and ensure they function as expected, such as Postman tests to verify the validity of all Express routes. A more thorough approach to test-driven development could also have been taken, as this would have meant writing tests with each feature introduction to avoid a large backlog.

9. Conclusions

Overall, the project was a moderate success, as all primary and secondary objectives were fulfilled. Additionally, the application cohesively combines several features presented in other variations of the *Racetrack* concept to produce its own variant. Further work could be done to improve the application's UX in line with Nielsen's (1994) usability heuristics and to ensure fairness across gameplay elements. Security in routing could also be significantly improved with better authentication.

10. References

AbdElazim, K., Moawad, R., & Elfakharany, E. (2020, February). *A Framework for Requirements Prioritization Process in Agile Software Development*. Journal of Physics: Conference Series, 1454, 012001. https://doi.org/10.1088/1742-6596/1454/1/012001.

Antonio, F. (1992). *IV.6 - FASTER LINE SEGMENT INTERSECTION*. In D. Kirk (Ed.), Graphics Gems III (IBM Version) (pp. 199–202). https://doi.org/10.1016/B978-0-08-050755-2.50045-2.

Bootstrap. (2026). *Bootstrap*. Retrieved April 27, 2026, from https://getbootstrap.com.

Boschloo, H. (2018, April). *Vector Racer*. Harmmade. Retrieved April 27, 2026, from https://harmmade.com/vectorracer/.

Catmull, E., & Rom, R. (1974). *A CLASS OF LOCAL INTERPOLATING SPLINES*. In R. E. Barnhill & R. F. Riesenfeld (Eds), Computer Aided Geometric Design (pp. 317–326). https://doi.org/10.1016/B978-0-12-079050-0.50020-5.

Chen, J.-C. (2003). *Dijkstra's Shortest Path Algorithm*. http://mizar.org/fm/2003-11/pdf11-3/graphsp.pdf.

Firebase. (2026). *Firebase*. Retrieved April 27, 2026, from https://firebase.google.com.

Flora, H., & Chande, S. (2013, July). *A Review and Analysis on Mobile Application Development Processes using Agile Methodologies*. International Journal of Research in Computer Science, 3, 8. https://doi.org/10.7815/ijorcs.34.2013.068.

Fredman, M. L., & Tarjan, R. E. (1987). Fibonacci heaps and their uses in improved network optimization algorithms. J. ACM, 34(3), 596–615. https://doi.org/10.1145/28869.28874.

Gardner, M. (1973). *Mathematical Games*, Scientific American Magazine Archives. http://www.scientificamerican.com/article/mathematical-games-1973-01/.

Graphics in 5 minutes. (2022-a, June 3). *Splines in 5 Minutes: Part 3 -- B-splines and 2D*. [Video] YouTube. https://www.youtube.com/watch?v=DLsqkWV6Cag.

Graphics in 5 minutes. (2022-b, June 3). *Splines in 5 Minutes: Part 2 -- Catmull-Rom and Natural Cubic Splines*. [Video] YouTube. https://www.youtube.com/watch?v=DLsqkWV6Cag.

Heričko, T. (2017, October). *A qualitative and quantitative comparison of PHP and Node.js for web development*.

Hormann, K., & Agathos, A. (2001). *The point in polygon problem for arbitrary polygons*. Computational Geometry, 20(3), 131–144. https://doi.org/10.1016/S0925-7721(01)00012-8.

Hunicke, R., & Chapman, V. (2004, January). *AI for dynamic difficulty adjustment in games*. Challenges in Game Artificial Intelligence AAAI Workshop, 2.

Hussain, A., Shakeel, H., Hussain, F., Uddin, N., & Ghouri, T. (2020, October). *Unity Game Development Engine: A Technical Survey*. University of Sindh Journal of Information and Communication Technology, 4.

Iyer, G. (2013). *Node.js: Event-driven Concurrency for Web Applications*. https://doi.org/10.13140/RG.2.1.2591.9849.

Jakubec, K., Polák, M., Nečaský, M., & Holubová, I. (2014). *Undo/Redo Operations in Complex Environments*. Procedia Computer Science, 32, 561–570. https://doi.org/10.1016/j.procs.2014.05.461.

javidx9. (2017, August 6). *Programming & Using Splines - Part#1*. [Video] YouTube. https://www.youtube.com/watch?v=9_aJGUTePYo.

JesterDay Studio. (2026). *GPixel*. Retrieved April 27, 2026, from https://jesterday.studio.

Kaboom. (n.d.). *Kaboom.js*. Retrieved April 27, 2026, from https://kaboomjs.com.

Kader, M., & Ahmed, R. (2025, November). *A Comprehensive Review on Full-Stack Development in the Cloud Era: Trends, Tools, and Best Practices*. Scientia. Technology, Science and Society, 2, 310–318. https://doi.org/10.59324/stss.2025.2(11).25.

Kodub. (2026). *PolyTrack*. Retrieved April 27, 2026, from https://www.kodub.com/apps/polytrack.

Kravchenko, T., Bogdanova, T., & Shevgunov, T. (2022, July). Ranking Requirements Using MoSCoW Methodology in Practice. https://doi.org/10.1007/978-3-031-09073-8_18.

Lazzaro, N. (2009). *Why We Play: Affect and the Fun of Games. Human Factors and Ergonomics*, 155–176. https://doi.org/10.1201/9781420088885.ch10.

Lucide. (2026). *Lucide*. Retrieved April 27, 2026, from https://lucide.dev.

Microsoft Azure. (2026). *Azure DevOps*. Retrieved April 27, 2026, from https://azure.microsoft.com/en-us/products/devops.

Mogra, M. (2025). *CS Project II Team Project Backlog - Boards*. Retrieved April 27, 2026, from https://dev.azure.com/MuhammadsOrganisation/CS%20Project%20II/_backlogs/backlog/.

MongoDB. (2026). *Atlas Database*. Retrieved April 27, 2026, from https://www.mongodb.com/products/platform/atlas-database.

Moroney, L. (2017, November). *Cloud Functions for Firebase*. https://doi.org/doi:10.1007/978-1-4842-2943-9_8.

Nielsen, J. (1994, April). *10 Usability Heuristics for User Interface Design*. Nielsen Norman Group. https://nngroup.com/articles/ten-usability/heuristics.

Node Package Manager. (2026-a, April). *Vitest*. Retrieved April 27, 2026, from https://npmjs.com/package/vitest.

Node Package Manager. (2026-b). *npm*. Retrieved April 27, 2026, from https://npmjs.com.

Ohyver, M., Moniaga, J. V., Sungkawa, I., Subagyo, B. E., & Chandra, I. A. (2019). *The Comparison Firebase Realtime Database and MySQL Database Performance using Wilcoxon Signed-Rank Test*. Procedia Computer Science, 157, 396–405. https://doi.org/10.1016/j.procs.2019.08.231.

OpenJFX. (n.d.). *JavaFX*. Retrieved April 27, 2026, from https://openjfx.io.

p5.js. (2026). *p5.js*. Retrieved April 27, 2026, from https://p5js.org.

Phaser Studio. (2026). *Phaser – A fast, fun and free open source HTML5 game framework*. Retrieved April 27, 2026, from https://phaser.io.

Python Software Foundation. (2026, April). *tkinter - Python interface to Tcl/Tk*. Retrieved April 27, 2026, from https://docs.python.org/3/library/tkinter.html.

Raul, S., Kevadiya, P., Khan, Z., & Pereira, J. (2025, July). A React.js-Based Approach for Enhancing Content Management and User Experience in Dynamic Web Applications. 89–94. https://doi.org/10.1109/ICoEIT63558.2025.11211620.

Russo, J., Bragg, L. A., & Russo, T. (2021). *How primary teachers use games to support their teaching of mathematics (Version 2)*. Deakin University. https://hdl.handle.net/10536/DRO/DU:30150385.

Sharma, K., & Sharma, M. (2026). *Growth and Evolution Of Typescript*. https://doi.org/10.13140/RG.2.2.18127.14246.

Šušter, I., Karabasevic, D., Stanujkić, D., Popovic, G., Veljić, A., & Marković, M. (2025, July). *DATA PROTECTION AND SECURITY IN MYSQL DATABASE*. 27.

Tailwind CSS. (2026). *Tailwind CSS*. Retrieved April 27, 2026, from https://trailwindcss.com.

Transum. (2026). *Inertia Racetrack*. Retrieved April 27, 2026, from https://www.transum.org/Maths/Game/Vectors/.

Ugwumba, N. (2025, December). *Master Backend Development with Node.js & Express: Complete Guide to REST APIs, Databases & Security*.

Williams, L. (2010). *Agile Software Development Methodologies and Practices*. In M. V. Zelkowitz (Ed.), Advances in Computers (pp. 1–44). https://doi.org/10.1016/S0065-2458(10)80001-4.

Wu, C., Huang, Y.-F., & Lee, J. (2015, April). *Comparisons between MongoDB and MS-SQL Databases on the TWC Website*. Journal of Software Engineering and Applications, 4, 35–41. https://doi.org/10.11648/j.ajsea.20150402.12.

Zagal, J. P., Nussbaum, M., & Rosas, R. (2000, October). *A Model to Support the Design of Multiplayer Games. Presence: Teleoperators and Virtual Environments*, 9(5), 448–462. https://doi.org/10.1162/105474600566943.

Zammetti, F. (2022). *Modern Full-Stack Development*. Apress. https://doi.org/10.1007/978-1-4842-8811-5.

Zheng, B. (2021). *Splines*. In K. Ikeuchi (Ed.), *Computer Vision: A Reference Guide* (pp. 1189–1191). https://doi.org/10.1007/978-3-030-63416-2_419.